



Faculty of Engineering at Cairo University

Ziad Kan 3aref Sala7

Ziad Montaser, Yousef Aref, Salah Mustafa

- 1 Contest
- 2 Mathematics
- 3 Data structures
- 4 Numerical
- 5 Number theory
- 6 Combinatorial
- 7 Graph
- 8 Geometry
- 9 Strings
- 10 Various

Contest (1)

template.cpp	28 lines
<pre>#include <bits/stdc++.h> using namespace std; #define all(x) (x).begin(), (x).end() using ll = long long int; using pii = pair<int, int>; using pll = pair<ll, ll>; using vi = vector<int>; using vl = vector<ll>; int dx[] = { 1, -1, 0 , 0 }; int dy[] = { 0, 0, 1 , -1 }; const int OO = (1 << 30), MOD = 1e9 + 7; const ll OOLL = (1ll << 60); const int N=1e5+7, M=5e3, MAXLOG=25; void solve() { }</pre>	
<pre>int main() { ios_base::sync_with_stdio(false); cin.tie(NULL); //freopen("input.txt", "r", stdin); //freopen("output.txt", "w", stdout); int t = 1; cin >> t; while (t--) solve(); }</pre>	
.bashrc	3 lines
<pre>alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++17 \ -fsanitize=undefined,address' xmodmap -e 'clear lock' -e 'keycode 66=less greater' #caps =◇</pre>	

.vimrc	6 lines
<pre>set cin aw ai is ts=4 sw=4 tm=50 nu noeb bg=dark ru cul sy on im jk <esc> im kj <esc> no ; : " Select region and then type :Hash to hash your selection. " Useful for verifying that there aren't mistypes. ca Hash w !cpp -dD -P -fpreprocessed \ tr -d '[:space:]' \ \ md5sum \ cut -c-6</pre>	
hash.sh	3 lines
<pre># Hashes a file, ignoring all whitespace and comments. Use for # verifying that code was correctly typed. cpp -dD -P -fpreprocessed tr -d '[:space:]' md5sum cut -c-6</pre>	
troubleshoot.txt	52 lines
<pre>Pre-submit: Write a few simple test cases if sample is not enough. Are time limits close? If so, generate max cases. Is the memory usage fine? Could anything overflow? Make sure to submit the right file.</pre>	
<pre>Wrong answer: Print your solution! Print debug output, as well. Are you clearing all data structures between test cases? Can your algorithm handle the whole range of input? Read the full problem statement again. Do you handle all corner cases correctly? Have you understood the problem correctly? Any uninitialized variables? Any overflows? Confusing N and M, i and j, etc.? Are you sure your algorithm works? What special cases have you not thought of? Are you sure the STL functions you use work as you think? Add some assertions, maybe resubmit. Create some testcases to run your algorithm on. Go through the algorithm for a simple case. Go through this list again. Explain your algorithm to a teammate. Ask the teammate to look at your code. Go for a small walk, e.g. to the toilet. Is your output format correct? (including whitespace) Rewrite your solution from the start or let a teammate do it.</pre>	
<pre>Runtime error: Have you tested all corner cases locally? Any uninitialized variables? Are you reading or writing outside the range of any vector? Any assertions that might fail? Any possible division by 0? (mod 0 for example) Any possible infinite recursion? Invalidated pointers or iterators? Are you using too much memory? Debug with resubmits (e.g. remapped signals, see Various).</pre>	
<pre>Time limit exceeded: Do you have any possible infinite loops? What is the complexity of your algorithm? Are you copying a lot of unnecessary data? (References) How big is the input and output? (consider scanf) Avoid vector, map. (use arrays/unordered_map) What do your teammates think about your algorithm?</pre>	
<pre>Memory limit exceeded: What is the max amount of memory your algorithm should need? Are you clearing all data structures between test cases?</pre>	

Mathematics (2)

2.1 Equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The extremum is given by $x = -b/2a$.

$$\begin{matrix} ax + by = e \\ cx + dy = f \end{matrix} \Rightarrow \begin{matrix} x = \frac{ed - bf}{ad - bc} \\ y = \frac{af - ec}{ad - bc} \end{matrix}$$

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

$$x_i = \frac{\det A'_i}{\det A}$$

where A'_i is A with the i 'th column replaced by b .

2.2 Recurrences

If $a_n = c_1a_{n-1} + \dots + c_ka_{n-k}$, and $r_1, \dots, r_k$ are distinct roots of $x^k - c_1x^{k-1} - \dots - c_k$, there are $d_1, \dots, d_k$ s.t.

$$a_n = d_1r_1^n + \dots + d_kr_k^n.$$

Non-distinct roots r become polynomial factors, e.g. $a_n = (d_1n + d_2)r^n$.

2.3 Trigonometry

$$\sin(v + w) = \sin v \cos w + \cos v \sin w$$

$$\cos(v + w) = \cos v \cos w - \sin v \sin w$$

$$\tan(v + w) = \frac{\tan v + \tan w}{1 - \tan v \tan w}$$

$$\sin v + \sin w = 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2}$$

$$\cos v + \cos w = 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2}$$

$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where V, W are lengths of sides opposite angles v, w .

$$a \cos x + b \sin x = r \cos(x - \phi)$$

$$a \sin x + b \cos x = r \sin(x + \phi)$$

where $r = \sqrt{a^2 + b^2}, \phi = \operatorname{atan2}(b, a)$.

2.4 Geometry

2.4.1 Triangles

Side lengths: a, b, c
Semiperimeter: $p = \frac{a + b + c}{2}$
Area: $A = \sqrt{p(p - a)(p - b)(p - c)}$
Circumradius: $R = \frac{abc}{4A}$
Inradius: $r = \frac{A}{p}$
Length of median (divides triangle into two equal-area triangles):
 $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$
Length of bisector (divides angles in two):

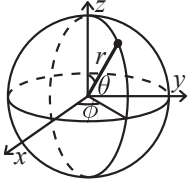
$$s_a = \sqrt{bc \left[1 - \left(\frac{a}{b + c} \right)^2 \right]}$$

Law of sines: $\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$
Law of cosines: $a^2 = b^2 + c^2 - 2bc \cos \alpha$
2.4.2 Quadrilaterals
Law of tangents: $\tan \frac{\alpha + \beta}{2} = \frac{a + b}{a - b} \tan \frac{\alpha - \beta}{2}$
With side lengths a, b, c, d , diagonals e, f , diagonals angle θ , area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin \theta = F \tan \theta = \sqrt{4e^2 f^2 - F^2}$$

2.4.3 Spherical coordinates

For cyclic quadrilaterals the sum of opposite angles is 180° ,
 $ef = ac + bd$, and $A = \sqrt{(p - a)(p - b)(p - c)(p - d)}$.



$$\begin{aligned} x &= r \sin \theta \cos \phi & r &= \sqrt{x^2 + y^2 + z^2} \\ y &= r \sin \theta \sin \phi & \theta &= \arccos(z / \sqrt{x^2 + y^2 + z^2}) \\ z &= r \cos \theta & \phi &= \operatorname{atan2}(y, x) \end{aligned}$$

2.5 Derivatives/Integrals

$$\begin{aligned} \frac{d}{dx} \arcsin x &= \frac{1}{\sqrt{1 - x^2}} & \frac{d}{dx} \arccos x &= -\frac{1}{\sqrt{1 - x^2}} \\ \frac{d}{dx} \tan x &= 1 + \tan^2 x & \frac{d}{dx} \arctan x &= \frac{1}{1 + x^2} \\ \int \tan ax &= -\frac{\ln |\cos ax|}{a} & \int x \sin ax &= \frac{\sin ax - ax \cos ax}{a^2} \\ \int e^{-x^2} &= \frac{\sqrt{\pi}}{2} \operatorname{erf}(x) & \int x e^{ax} dx &= \frac{e^{ax}}{a^2} (ax - 1) \end{aligned}$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.6 Sums

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c - 1}, c \neq 1$$

$$\begin{aligned} 1 + 2 + 3 + \cdots + n &= \frac{n(n + 1)}{2} \\ 1^2 + 2^2 + 3^2 + \cdots + n^2 &= \frac{n(2n + 1)(n + 1)}{6} \\ 1^3 + 2^3 + 3^3 + \cdots + n^3 &= \frac{n^2(n + 1)^2}{4} \\ 1^4 + 2^4 + 3^4 + \cdots + n^4 &= \frac{n(n + 1)(2n + 1)(3n^2 + 3n - 1)}{30} \end{aligned}$$

2.7 Series

$$\begin{aligned} e^x &= 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots, \quad (-\infty < x < \infty) \\ \ln(1 + x) &= x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \cdots, \quad (-1 < x \leq 1) \\ \sqrt{1 + x} &= 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \cdots, \quad (-1 \leq x \leq 1) \\ \sin x &= x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots, \quad (-\infty < x < \infty) \\ \cos x &= 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots, \quad (-\infty < x < \infty) \end{aligned}$$

2.8 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x . It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x x p_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y ,

$$V(aX + bY) = a^2V(X) + b^2V(Y).$$

2.8.1 Discrete distributions
Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\operatorname{Bin}(n, p)$, $n = 1, 2, \dots$, $0 \leq p \leq 1$.

$$p(k) = \binom{n}{k} p^k (1 - p)^{n - k}$$

$$\mu = np, \sigma^2 = np(1 - p)$$

$\operatorname{Bin}(n, p)$ is approximately $\operatorname{Po}(np)$ for small p .
First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $\operatorname{Fs}(p)$, $0 \leq p \leq 1$.

$$p(k) = p(1 - p)^{k - 1}, k = 1, 2, \dots$$

$$\mu = \frac{1}{p}, \sigma^2 = \frac{1 - p}{p^2}$$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $\operatorname{Po}(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda} \frac{\lambda^k}{k!}, k = 0, 1, 2, \dots$$

$$\mu = \lambda, \sigma^2 = \lambda$$

2.8.2 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $U(a,b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$
$$\mu = \frac{a+b}{2}, \sigma^2 = \frac{(b-a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is $\text{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$
$$\mu = \frac{1}{\lambda}, \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ then

$$aX_1 + bX_2 + c \sim \mathcal{N}(\mu_1 + \mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

2.9 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. Let $X_1, X_2, \dots$ be a sequence of random variables generated by the Markov process. Then there is a transition matrix $\mathbf{P} = (p_{ij})$, with $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, and $\mathbf{p}^{(n)} = \mathbf{P}^n \mathbf{p}^{(0)}$ is the probability distribution for X_n (i.e., $p_i^{(n)} = \Pr(X_n = i)$), where $\mathbf{p}^{(0)}$ is the initial distribution.

π is a stationary distribution if $\pi = \pi \mathbf{P}$. If the Markov chain is *irreducible* (it is possible to get to any state from any state), then $\pi_i = \frac{1}{\mathbb{E}(T_i)}$ where $\mathbb{E}(T_i)$ is the expected time between two visits in state i . π_j / π_i is the expected number of visits in state j between two visits in state i .

For a connected, undirected and non-bipartite graph, where the transition probability is uniform among all neighbors, π_i is proportional to node i 's degree.

A Markov chain is *ergodic* if the asymptotic distribution is independent of the initial distribution. A finite Markov chain is ergodic iff it is irreducible and *aperiodic* (i.e., the gcd of cycle lengths is 1). $\lim_{k \rightarrow \infty} \mathbf{P}^k = \mathbf{1}\pi$.

A Markov chain is an A-chain if the states can be partitioned into two sets $\mathbf{A}$ and $\mathbf{G}$, such that all states in $\mathbf{A}$ are absorbing ($p_{ii} = 1$), and all states in $\mathbf{G}$ leads to an absorbing state in $\mathbf{A}$. The probability for absorption in state $i \in \mathbf{A}$, when the initial state is j , is $a_{ij} = p_{ij} + \sum_{k \in \mathbf{G}} a_{ik} p_{kj}$. The expected time until absorption, when the initial state is i , is $t_i = 1 + \sum_{k \in \mathbf{G}} p_{ki} t_k$.

Data structures (3)

OrderedSet.h

Description: A set with support for finding the n'th element, and finding the index of an element. To get a map, change null_type. Use less_equal or greater_equal to turn it into a MultiSet. NOTE in case of using a multi set: - erase function is corrupted and you should use the below utility - lower_bound and upper_bound are reversed

Time: $\mathcal{O}(\log N)$ bf6067, 14 lines

```
#include<ext/pb_ds/assoc_container.hpp>
#include<ext/pb_ds/tree_policy.hpp>

using namespace __gnu_pbds;

typedef tree<ll , null_type, less<ll>, rb_tree_tag,
            tree_order_statistics_node_update> OrderedSet;
// find_by_order : element at index i
// order_of_key  : how many elements < A for less , and how many
                  ele > A for greater

void erase(OrderedSet &t, int v){
    int rank = t.order_of_key(v);
    auto it = t.find_by_order(rank);
    t.erase(it);
}
```

HashMap.h

Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

```
d77092, 7 lines
#include <bits/extc++.h>
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
    const uint64_t C = 11(4e18 * acos(0)) | 71;
    ll operator()(ll x) const { return __builtin_bswap64(x*C); }
};
__gnu_pbds::gp_hash_table<ll,int,chash> h({}, {}, {}, {}, {1<<16});
```

SegmentTree.h

Description: Zero-indexed array-based Segment Tree. Bounds are inclusive to the left and exclusive to the right $[l, r)$. Computes range sum by default. Can be changed by modifying the 'Node' struct and 'merge' function.

Usage: SegmentTree st(sz(v)); st.build(v); st.set(idx, val); st.get_range(l, r);

Time: $\mathcal{O}(\log N)$ for queries and updates, $\mathcal{O}(N)$ to build. 1711f1, 75 lines

```
struct Node {
    long long sum;
    Node() : sum(0){}
    Node(int x) : sum(x){}
};

struct SegTree {
    int tree_size;
    vector<Node> SegData;
    SegTree(int n) {
```

```
tree_size = 1;
while (tree_size < n) tree_size <= 1;
SegData.assign(2 * tree_size, Node());
}

Node merge(const Node & lf, const Node & ri) {
    Node ans = Node();
    ans.sum = lf.sum + ri.sum;
    return ans;
}

void build(const vector<int> & arr, int node, int lx, int rx) {
    rx) {
        if(rx - lx == 1) {
            if(lx < arr.size())
                SegData[node] = Node(arr[lx]);
            return;
        }

        int mid = (lx + rx) / 2;
        build(arr, 2 * node + 1, lx, mid);
        build(arr, 2 * node + 2, mid, rx);

        SegData[node] = merge(SegData[2 * node + 1], SegData[2 * node + 2]);
    }
}

void build(const vector<int> & arr) {
    build(arr, 0, 0, tree_size);
}

void set(int idx, int val, int node, int lx, int rx) {
    if(rx - lx == 1) {
        SegData[node].sum = val;
        return;
    }

    int mid = (lx + rx) / 2;
    if(idx < mid)
        set(idx, val, 2 * node + 1, lx, mid);
    else
        set(idx, val, 2 * node + 2, mid, rx);

    SegData[node] = merge(SegData[2 * node + 1], SegData[2 * node + 2]);
}
// 0 indexed
void set(int idx, int val) {
    set(idx, val, 0, 0, tree_size);
}

Node get_range(int l, int r, int node, int lx, int rx) {
    if(lx >= r || rx <= l)
        return Node();
    if(lx >= l && rx <= r)
        return SegData[node];

    int mid = (lx + rx) / 2;
    Node lf = get_range(l, r, 2 * node + 1, lx, mid);
    Node ri = get_range(l, r, 2 * node + 2, mid, rx);

    return merge(lf, ri);
}

// r is not inclusive [l, r)
long long get_range(int l, int r) {
    return get_range(l, r, 0, 0, tree_size).sum;
}
};
```

SegmentTreeSmall.h

Description: Zero-indexed max-tree. Bounds are inclusive to the left and exclusive to the right. Can be changed by modifying T, f and unit.

Time: $\mathcal{O}(\log N)$

0f4bdb, 19 lines

```
struct Tree {
    typedef int T;
    static constexpr T unit = INT_MIN;
    T f(T a, T b) { return max(a, b); } // (any associative fn)
    vector<T> s; int n;
    Tree(int n = 0, T def = unit) : s(2*n, def), n(n) {}
    void update(int pos, T val) {
        for (s[pos += n] = val; pos /= 2;)
            s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
    }
    T query(int b, int e) { // query [b, e)
        T ra = unit, rb = unit;
        for (b += n, e += n; b < e; b /= 2, e /= 2) {
            if (b % 2) ra = f(ra, s[b++]);
            if (e % 2) rb = f(s[--e], rb);
        }
        return f(ra, rb);
    }
};
```

LazySegmentTree.h

Description: Array-based Lazy Segment Tree. Supports range add, range set, and range sum queries. 0-indexed API, range queries are half-open $[l, r)$.

Usage: SegTree st(sz(v)); st.build(v); st.add(l, r, val); st.set(l, r, val); st.get.range(l, r);

Time: $\mathcal{O}(\log N)$ for queries and updates, $\mathcal{O}(N)$ for building

75a74e, 276 lines

```
struct Node {
    long long sum = 0;
    long long lazy_set, lazy_add = 0;
    bool is_lazy_set = 0, is_lazy_add = 0;

    Node(int x) : sum(x) {}
    Node() {}

    void add(int val, int lx, int rx) {
        sum += (rx - lx) * val;
        lazy_add += val;
        is_lazy_add = 1;
    }

    void set(int val, int lx, int rx) {
        sum = (rx - lx) * val;
        lazy_set = val;
        is_lazy_set = 1;
        lazy_add = 0;
        is_lazy_add = 0;
    }
};

struct SegTree {
    int tree_size;
    vector<Node> SegData;
    SegTree(int n) {
        tree_size = 1;
        while (tree_size < n) tree_size <<= 1;
        SegData.assign(2 * tree_size, Node());
    }

    Node merge(const Node & lf, const Node & ri) {
        Node ans = Node();
        ans.sum = lf.sum + ri.sum;

        return ans;
    }
};
```

```
void build(const vector<int> & arr, int node, int lx, int rx) {
    if(rx - lx == 1) {
        if(lx < arr.size())
            SegData[node] = Node(arr[lx]);
        return;
    }

    int mid = (lx + rx) / 2;
    build(arr, 2 * node + 1, lx, mid);
    build(arr, 2 * node + 2, mid, rx);

    SegData[node] = merge(SegData[2 * node + 1], SegData[2 * node + 2]);
}

void build(const vector<int> & arr) {
    build(arr, 0, 0, tree_size);
}

void propagate(int node, int lx, int rx) {
    if(rx - lx == 1)
        return;

    int mid = (lx + rx) / 2;

    if(SegData[node].is_lazy_set) {
        SegData[2 * node + 1].set(SegData[node].lazy_set, lx, mid);
        SegData[2 * node + 2].set(SegData[node].lazy_set, mid, rx);

        SegData[node].lazy_set = 0;
        SegData[node].is_lazy_set = 0;
    }

    if(SegData[node].is_lazy_add) {
        SegData[2 * node + 1].add(SegData[node].lazy_add, lx, mid);
        SegData[2 * node + 2].add(SegData[node].lazy_add, mid, rx);

        SegData[node].lazy_add = 0;
        SegData[node].is_lazy_add = 0;
    }
}

void set(int l, int r, int val, int node, int lx, int rx) {
    propagate(node, lx, rx);
    if(lx >= r || rx <= l)
        return;
    if(lx >= l && rx <= r) {
        SegData[node].set(val, lx, rx);
        return;
    }

    int mid = (lx + rx) / 2;
    set(l, r, val, 2 * node + 1, lx, mid);
    set(l, r, val, 2 * node + 2, mid, rx);
    SegData[node] = merge(SegData[2 * node + 1], SegData[2 * node + 2]);
}

// 0 indexed
void set(int l, int r, int val) {
    set(l, r, val, 0, 0, tree_size);
}

void add(int l, int r, int val, int node, int lx, int rx) {
    propagate(node, lx, rx);
```

```
    if(lx >= r || rx <= l)
        return;
    if(lx >= l && rx <= r) {
        SegData[node].add(val, lx, rx);
        return;
    }

    int mid = (lx + rx) / 2;
    add(l, r, val, 2 * node + 1, lx, mid);
    add(l, r, val, 2 * node + 2, mid, rx);
    SegData[node] = merge(SegData[2 * node + 1], SegData[2 * node + 2]);
}

// 0 indexed
void add(int l, int r, int val) {
    add(l, r, val, 0, 0, tree_size);
}

Node get_range(int l, int r, int node, int lx, int rx) {
    propagate(node, lx, rx);
    if(lx >= r || rx <= l)
        return Node();
    if(lx >= l && rx <= r)
        return SegData[node];

    int mid = (lx + rx) / 2;
    Node lf = get_range(l, r, 2 * node + 1, lx, mid);
    Node ri = get_range(l, r, 2 * node + 2, mid, rx);

    return merge(lf, ri);
}

// r is not inclusive [l, r)
long long get_range(int l, int r) {
    return get_range(l, r, 0, 0, tree_size).sum;
}

};

// ziad template
struct SegmentTree{
    int size;

#define SegmentType int
#define LazyType int
    SegmentType NUTRAL_ELEMENT = 0;
    LazyType NO_OPERATION = 0;

#define L (2*i+1)
#define R (2*i+2)

    vector<SegmentType> a;
    vector<LazyType> lazy;

    void AccumlateOperation(LazyType &a, LazyType b, int len = 1){
        a += b;
    }

    void ModifyOperation(SegmentType &a, LazyType b, int len = 1){
        a = a + b;
    }

    SegmentType CalcOperation(SegmentType a, SegmentType b){
        return merge(a, b);
    }

    SegmentType merge(SegmentType a, SegmentType b){
        return max(a, b);
    }
};
```

```
    }

    void build(vector<SegmentType> &b){
        size = 1;
        while(size < b.size()) size *= 2;

        a.resize(size * 2, NUTRAL_ELEMENT);
        lazy.resize(size * 2, NO_OPERATION);

        for (int i = 0; i < b.size(); i++)
        {
            a[i+size-1] = b[i];
        }
        build(0, 0, size);
    }

    void build(int i, int l, int r){
        if(i >= a.size()) return;
        if(r - l == 1) return;

        int mid = (l + r) / 2;
        build(2 * i + 1, l, mid);
        build(2 * i + 2, mid, r);

        a[i] = merge(a[L], a[R]);
    }

    void Propogate(int i, int l, int r){
        if(lazy[i] == NO_OPERATION) return;

        ModifyOperation(a[i], lazy[i], (r - l));

        if(r - l > 1){
            AccumlateOperation(lazy[L], lazy[i]);
            AccumlateOperation(lazy[R], lazy[i]);
        }
        lazy[i] = NO_OPERATION;
    }

    SegmentType CalculateRec(int i, int l, int r, int lx, int rx){
        rx){
            Propogate(i, l, r);

            if(l >= rx || r <= lx) return NUTRAL_ELEMENT;
            if(l >= lx && r <= rx){
                return a[i];
            }

            int mid = (l+r) / 2;
            return CalcOperation(CalculateRec(L, l, mid, lx, rx),
                                CalculateRec(R, mid, r, lx, rx));
        }

    SegmentType Calculate(int l, int r){
        return CalculateRec(0, 0, size, l, r);
    }

    void Modify(int i, int l, int r, int lx, int rx, LazyType vx){
        Propogate(i, l, r);

        if(l >= rx || r <= lx) return;
        if(l >= lx && r <= rx) {
            AccumlateOperation(lazy[i], vx);

            Propogate(i, l, r);
            return;
        }
    }
```

```
        int mid = (l+r) / 2;
        Modify(L, l, mid, lx, rx, vx);
        Modify(R, mid, r, lx, rx,vx);

        a[i] = CalcOperation(a[L], a[R]);
    }

    void Modify(int l, int r, LazyType v){
        Modify(0, 0, size, l, r, v);
    }

    SegmentType Find(int i, int l, int r, int lx, int rx, int vx){
        Propogate(i, l, r);

        if(l >= rx || r <= lx) return -1;
        if(l >= lx && r <= rx && a[i] < vx) {
            return -1;
        }

        if(r - l == 1){
            if(a[i] >= vx) return 1;
            else return -1;
        }

        int mid = (l+r) / 2;

        auto left = Find(L, l, mid, lx, rx, vx);
        if(left != -1) return left;

        auto right = Find(R, mid, r, lx, rx, vx);
        return right;
    }

    SegmentType Find(int l, int r, int k){
        return Find(0, 0, size, l, r, k);
    }

    #undef SegmentType
    #undef LazyType
    #undef L
    #undef R
};
```

MergeSortTree.h

Description: Merge Sort Tree. Answers 2D range queries online. Default implementation returns the number of elements strictly less than x in half-open range $[l, r)$. To change the query condition, replace the lower_bound call in less() based on this cheat sheet: 1. Strictly Greater than x: $v.end() - upper_bound(all(v), x)$ 2. Greater than or Equal to x: $v.end() - lower_bound(all(v), x)$ 3. Strictly Less than x: $lower_bound(all(v), x) - v.begin()$ 4. Less than or Equal to x: $upper_bound(all(v), x) - v.begin()$ 5. Equal to x: $upper_bound(all(v), x) - lower_bound(all(v), x)$ 6. Not Equal to x: $sz(v) - (upper_bound(all(v), x) - lower_bound(all(v), x))$
Time: $\mathcal{O}(N \log N)$ to build, $\mathcal{O}(\log^2 N)$ per query.

```
struct SegTree {
    int tree_size;
    vector<vector<int>> SegData;
    SegTree(int n) {
        tree_size = 1;
        while (tree_size < n) tree_size <= 1;
        SegData.assign(2 * tree_size, {});
    }

    vector<int> merge(const vector<int> &lf, const vector<int>
        &ri) {
```

```
        int n = lf.size(), m = ri.size();
        vector<int> ans(n + m);

        int i = 0, j = 0, k = 0;
        while (i < n && j < m) {
            if(lf[i] < ri[j])
                ans[k++] = lf[i++];
            else
                ans[k++] = ri[j++];
        }

        while (i < n)
            ans[k++] = lf[i++];
        while (j < m)
            ans[k++] = ri[j++];

        return ans;
    }

    void build(const vector<int> &arr, int node, int lx, int rx) {
        rx) {
            if(rx - lx == 1) {
                if(lx < arr.size())
                    SegData[node] = {arr[lx]};
                return;
            }

            int mid = (lx + rx) / 2;
            build(arr, 2 * node + 1, lx, mid);
            build(arr, 2 * node + 2, mid, rx);

            SegData[node] = merge(SegData[2 * node + 1], SegData[2
                * node + 2]);
        }

    void build(const vector<int> &arr) {
        build(arr, 0, 0, tree_size);
    }

    int less(int l, int r, int x, int node, int lx, int rx) {
        if(lx >= r || rx <= l)
            return 0;
        if(lx >= l && rx <= r) {
            return lower_bound(SegData[node].begin(), SegData[
                node].end(), x) - SegData[node].begin();
            // greater -> return SegData[node].end() -
                upper_bound(SegData[node].begin(), SegData[
                    node].end(), x);
        }

        int mid = (lx + rx) / 2;
        int lf = less(l, r, x, 2 * node + 1, lx, mid);
        int ri = less(l, r, x, 2 * node + 2, mid, rx);
        return lf + ri;
    }

    int less(int l, int r, int x) {
        return less(l, r, x, 0, 0, tree_size);
    }
};
```

DynamicSegTree.h

Description: Dynamic (pointer-based) Segment Tree. Allocates nodes lazily to save memory, allowing range queries and point updates on large intervals. Uses half-open intervals $[l, r)$.
Time: $\mathcal{O}(\log N)$ for updates and queries.

```
struct TreeNode {
    int sum = 0;
    TreeNode *left = nullptr, *right = nullptr;
```

```
void update(int val) {
    sum += val;
}

using Node = TreeNode*;

struct DynamicSegTree {
    int n;
    Node root;

    DynamicSegTree(int sz) {
        n = sz;
        root = new TreeNode();
    }

    void update(int idx, int &val, Node &node, int lx, int rx)
    {
        if(rx - lx == 1) {
            node->update(val);
            return;
        }

        int mid = (lx + rx) / 2;
        if(idx < mid) {
            if (node->left == nullptr) node->left = new
                TreeNode();
            update(idx, val, node->left, lx, mid);
        }
        else {
            if (node->right == nullptr) node->right = new
                TreeNode();
            update(idx, val, node->right, mid, rx);
        }

        node->sum = ((node->left == nullptr) ? 0 : node->left->
            sum) +
            ((node->right == nullptr) ? 0 : node->right
                ->sum);
    }

    void update(int idx, int val) {
        update(idx, val, root, 0, n);
    }

    ll query(int l, int r, Node &node, int lx, int rx) {
        if(node == nullptr || lx >= r || rx <= l) return 0;
        if (lx >= l && rx <= r) return node->sum;

        int mid = (lx + rx) / 2;
        ll L = query(l, r, node->left, lx, mid);
        ll R = query(l, r, node->right, mid, rx);
        return L + R;
    }

    ll query(int l, int r) {
        return query(l, r, root, 0, n);
    }
};
```

SubMatrix.h

Description: Calculate submatrix sums quickly, given upper-left and lower-right corners (half-open).

Usage: SubMatrix<int> m(matrix);
m.sum(0, 0, 2, 2); // top left 4 elements

Time: $\mathcal{O}(N^2 + Q)$

```
template<class T>
struct SubMatrix {
    vector<vector<T>> p;
    SubMatrix(vector<vector<T>>& v) {
```

```
int R = sz(v), C = sz(v[0]);
p.assign(R+1, vector<T>(C+1));
rep(r,0,R) rep(c,0,C)
    p[r+1][c+1] = v[r][c] + p[r][c+1] + p[r+1][c] - p[r][c];
}
T sum(int u, int l, int d, int r) {
    return p[d][r] - p[d][l] - p[u][r] + p[u][l];
}
};
```

Matrix.h

Description: Basic operations on square matrices.

Usage: Matrix<int, 3> A;
A.d = {{ {{1,2,3}}, {{4,5,6}}, {{7,8,9}} }};
array<int, 3> vec = {1,2,3};
vec = (A*N) * vec;

```
template<class T, int N> struct Matrix {
    typedef Matrix M;
    array<array<T, N>, N> d{};
    M operator*(const M& m) const {
        M a;
        rep(i,0,N) rep(j,0,N)
            rep(k,0,N) a.d[i][k] += d[i][j] * m.d[j][k];
        return a;
    }
    array<T, N> operator*(const array<T, N>& vec) const {
        array<T, N> ret{};
        rep(i,0,N) rep(j,0,N) ret[i] += d[i][j] * vec[j];
        return ret;
    }
    M operator^(ll p) const {
        assert(p >= 0);
        M a, b(*this);
        rep(i,0,N) a.d[i][i] = 1;
        while (p) {
            if (p&1) a = a*b;
            b = b*b;
            p >>= 1;
        }
        return a;
    }
};
```

Treap.h

Description: Implicit Treap (Cartesian Tree) acting as a sequential container. Supports range sum, range add (lazy propagation), range reverse, and cyclic shifts. Maintains parent pointers to allow finding the rank (index) of a node.

Time: $\mathcal{O}(\log N)$ for splits, joins, and range operations.

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().
    count());
#define getrand(l, r) uniform_int_distribution<long long>(l, r)
    (rng)

struct TreapNode {
    int sz = 1;
    ll p = getrand(1, 1e18);

    ll sum = 0, lazy = 0, val;
    bool is_lazy = 0;
    bool rev = 0;

    TreapNode *l = NULL, *r = NULL, *par = NULL;

    TreapNode(int a): val(a), sum(a){}
};
```

```
using Treap = TreapNode*;

int size(Treap t) {
    return t ? t->sz : 0;
}

ll sum(Treap t) {
    return t ? t->sum : 0;
}

void apply_lazy(Treap t, ll val) {
    if(!t) return;
    t->val += val;
    t->sum += t->sz * val;
    t->lazy += val;
    t->is_lazy = 1;
}

void apply_rev(Treap t) {
    if(!t) return;
    swap(t->l, t->r);
    t->rev ^= 1;
}

void prop(Treap t) {
    if(!t) return;

    if(t->rev) {
        apply_rev(t->l);
        apply_rev(t->r);
        t->rev = 0;
    }

    if(t->is_lazy) {
        apply_lazy(t->l, t->lazy);
        apply_lazy(t->r, t->lazy);
        t->lazy = 0;
        t->is_lazy = 0;
    }
}

Treap recalc(Treap t) {
    prop(t->l), prop(t->r);
    t->sz = size(t->l) + 1 + size(t->r);
    t->sum = sum(t->l) + t->val + sum(t->r);
    return t;
}

Treap merge(Treap l, Treap r) {
    if(!l || !r) return r ? r : l;
    prop(l), prop(r);

    if(l->p < r->p) {
        Treap ri = merge(l->r, r);
        l->r = ri;
        if(ri != NULL) ri->par = l;
        return recalc(l);
    }

    Treap lf = merge(l, r->l);
    r->l = lf;
    if(lf != NULL) lf->par = r;

    return recalc(r);
}

array<Treap, 2> split(Treap t, int cnt) {
    if(!t) return {NULL, NULL};
    prop(t);
```

```

    if(size(t->l) >= cnt) {
        auto [left, right] = split(t->l, cnt);
        t->l = right;

        if(left != NULL) left->par = NULL;
        if(right != NULL) right->par = t;

        return {left, recalc(t)};
    }

    auto [left, right] = split(t->r, cnt - size(t->l) - 1);
    t->r = left;

    if(left != NULL) left->par = t;
    if(right != NULL) right->par = NULL;

    return {recalc(t), right};
}

Treap left_shift(Treap t, int l, int r) {
    auto [a, b] = split(t, r);
    auto [c, d] = split(a, l - 1);
    auto [e, f] = split(d, l);
    return merge(merge(c, f), merge(e, b));
}

Treap add_range(Treap t, int l, int r, int val) {
    auto [a, b] = split(t, r);
    auto [c, d] = split(a, l - 1);

    apply_lazy(d, val);

    return merge(merge(c, d), b);
}

Treap get_range(Treap t, int l, int r) {
    auto [a, b] = split(t, r);
    auto [c, d] = split(a, l - 1);

    cout << d->sum << '\n';

    return merge(merge(c, d), b);
}

Treap reverse_range(Treap t, int l, int r) {
    auto [a, b] = split(t, r);
    auto [c, d] = split(a, l - 1);

    apply_rev(d);
    return merge(merge(c, d), b);
}

void print(Treap t) {
    if(!t) return;
    prop(t);
    print(t->l);
    cout << t->val << ' ';
    print(t->r);
}

int get_idx_of_value(Treap t) {
    if(!t) return 0;

    int idx = size(t->l) + 1;
    Treap curr = t;

    while (curr->par) {
        if(curr == curr->par->r)
            idx += size(curr->par->l) + 1;
    }

```

```

        curr = curr->par;
    }

    return idx;
}

Treap get_val(Treap t, int idx) {
    auto [a, b] = split(t, idx);
    auto [c, d] = split(a, idx - 1);

    cout << d->val << '\n';

    return merge(merge(c, d), b);
}

```

FenwickTree.h

Description: Computes partial sums $a[0] + a[1] + \dots + a[\text{pos} - 1]$, and updates single elements $a[i]$, taking the difference between the old and new value, and is 1-indexed.

Time: Both operations are $\mathcal{O}(\log N)$.

f13e04, 40 lines

```

// 1-indexed
struct BIT {

    int n;
    vector<long long> b;

    BIT(int _n) {
        n = _n;
        b.assign(n + 1, 0);
    }

    long long get(int idx) {
        long long res = 0;
        while (idx > 0) {
            res += b[idx];
            idx -= idx & -idx;
        }
        return res;
    }

    void add(int idx, int v) {
        while (idx <= n) {
            b[idx] += v;
            idx += idx & -idx;
        }
    }

    long long get(int l, int r) {
        return get(r) - get(l - 1);
    }

    long long get_idx(int idx) {
        return get(idx) - get(idx - 1);
    }

    void set(int idx, int v) {
        int old = get_idx(idx);
        add(idx, -old + v);
    }
};

```

FenwickTree2d.h

Description: Offline 2D Fenwick Tree. Computes rectangle sums and updates single elements. Requires all point coordinates to be updated/queried to be known in advance and passed to the constructor. Automatically deduplicates coordinates to save memory. 0-indexed API.

Memory: $\mathcal{O}(M \log N)$

Time: $\mathcal{O}(\log^2 N)$ per query/update, $\mathcal{O}(M \log M + M \log^2 N)$ initialization where M is the number of points.

"FenwickTree.h"

270f1e, 69 lines

```

struct FT2 {
    vector<vi> ys; vector<FT> ft;
    FT2(int limx) : ys(limx) {}
    void fakeUpdate(int x, int y) {
        for (; x < sz(ys); x |= x + 1) ys[x].push_back(y);
    }
    void init() {
        for (vi& v : ys) sort(all(v)), ft.emplace_back(sz(v));
    }
    int ind(int x, int y) {
        return (int)(lower_bound(all(ys[x]), y) - ys[x].begin());
    }
    void update(int x, int y, ll dif) {
        for (; x < sz(ys); x |= x + 1)
            ft[x].update(ind(x, y), dif);
    }
    ll query(int x, int y) {
        ll sum = 0;
        for (; x; x &= x - 1)
            sum += ft[x-1].query(ind(x-1, y));
        return sum;
    }
};

template <typename T> class OfflineBIT2D {
private:
    const int n;
    vector<vector<int>> vals;
    vector<vector<T>> bit;
    int ind(const vector<int> &v, int x) {
        return upper_bound(begin(v), end(v), x) - begin(v) - 1;
    }

public:
    OfflineBIT2D(int n, vector<array<int, 2>> &todo) : n(n),
        vals(n + 1), bit(n + 1) {
        sort(begin(todo), end(todo),
            [](const array<int, 2> &a, const array<int, 2> &b)
                -> bool {
                    return a[1] < b[1];
                });

        for (int i = 1; i <= n; i++) { vals[i].push_back(0); }
        for (auto [r, c] : todo) {
            r++, c++;
            for (; r <= n; r += r & -r) {
                if (vals[r].back() != c) { vals[r].push_back(c); }
            }
        }
        for (int i = 1; i <= n; i++) { bit[i].resize(vals[i].size()); }

        void add(int r, int c, T val) {
            r++, c++;
            for (; r <= n; r += r & -r) {
                int i = ind(vals[r], c);
                for (; i < bit[r].size(); i += i & -i) { bit[r][i] += val; }
            }
        }

        T rect_sum(int r, int c) {
            r++, c++;
            T sum = 0;
            for (; r > 0; r -= r & -r) {
                int i = ind(vals[r], c);
                for (; i > 0; i -= i & -i) { sum += bit[r][i]; }
            }
        }
    };
};

```



```
    }
    return sum;
}
T rect_sum(int r1, int c1, int r2, int c2) {
    return rect_sum(r2, c2) - rect_sum(r2, c1 - 1) -
        rect_sum(r1 - 1, c2) +
        rect_sum(r1 - 1, c1 - 1);
}
};
```

SparseTable.h
Description: Sparse Table for Range Minimum Queries (or any idempotent operation). Answers range queries in $O(1)$ time for idempotent operations (like min, max, gcd). Modify the ‘merge’ function to change the operation.
Usage: ST st(values);
st.get(l, r); // inclusive [l, r]
Time: $\mathcal{O}(|V|\log|V|)$ initialization, $\mathcal{O}(1)$ per query.

114475, 32 lines

```
struct ST{
    vector<vector<int>>> data;
    vector<int> logs;

    int merge(int &lf, int &ri) {
        return min(lf, ri);
    }

    ST(vector<int> &arr) {
        int n = arr.size();
        logs.assign(n + 1, 0);

        for (int i = 2; i <= n; ++i)
            logs[i] = logs[i / 2] + 1;

        data.assign(logs[n] + 1, vector<int>(n));
        data[0] = arr;

        for (int i = 1; i <= logs[n]; ++i) {
            int l = 1 << i;
            for (int j = 0; j + 1 <= n; ++j) {
                data[i][j] = merge(data[i - 1][j], data[i - 1][
                    j + (1 >> i)]);
            }
        }

        int get(int l, int r) {
            int len = r - l + 1;
            int level = logs[len];
            return merge(data[level][l], data[level][r - (1 <<
                level) + 1]);
        }
    }
};
```

MoQueries.h
Description: Mo’s algorithm for offline array range queries. Answers queries by sorting them in a cache-friendly way and moving interval endpoints. Modify ‘add()’ and ‘rem()’ to track the current state, and update ‘ans’.
Time: $\mathcal{O}(N\sqrt{Q})$

cb9673, 50 lines

```
struct Query {
    int l, r, idx;
};

struct Mo {
    int n, sq;
    vector<int> arr;
    int ans;
    vector<int> freq;
    Mo(vector<int> &v, int q) {
```

```
        arr = v;
        n = v.size();
        sq = sqrt(n) + 1;
        freq.assign(1e6 + 1, 0);
        ans = 0;
    }

    void add(int idx) {

    }

    void rem(int idx) {

    }

    void solve(vector<Query> &q) {

        sort(q.begin(), q.end(), [&](auto &lf, auto &ri) {
            if(lf.l / sq == ri.l / sq)
                return (lf.l / sq) & 1 ? lf.r > ri.r : lf.r <
                    ri.r;
            return lf.l / sq < ri.l / sq;
        });

        int l = q[0].l, r = q[0].l;
        add(l);

        vector<ll> res(q.size());
        for(auto [lq, rq, idx] : q) {
            while (r < rq) add(++r);
            while (l < lq) rem(l++);
            while (l > lq) add(--l);
            while (r > rq) rem(r--);

            res[idx] = ans;
        }

        for (int i = 0; i < res.size(); ++i) {
            cout << res[i] << '\n';
        }
    }
};
```

MoQueries-rollback.h
Description: Mo’s algorithm with Rollback for offline array range queries. Used when adding elements is easy but removing is hard. Modify ‘add()’ to track the state and push to ‘history’ if ‘record’ is true.
Time: $\mathcal{O}(N\sqrt{Q})$

a0a6f4, 92 lines

```
struct Query {
    int l, r, idx;
};

struct Mo {
    int n, sq;
    vector<int> arr;
    int ans;
    vector<int> freq;

    struct State {
        int old_ans;
        int pos;
        int val;
    };
    vector<State> history;

    Mo(vector<int> &v) {
        arr = v;
        n = v.size();
        sq = sqrt(n) + 1;
```

```
        freq.assign(n + 1, 0);
        ans = 0;
    }

    void add(int idx, bool record = false) {
        if(record) {
            history.push_back({ans, arr[idx], freq[arr[idx]]});
        }

        // edit
    }

    void rollback() {
        while (!history.empty()) {
            auto [old_ans, pos, val] = history.back();
            history.pop_back();
            ans = old_ans;
            freq[pos] = val;
        }
    }

    void reset() {
        fill(freq.begin(), freq.end(), 0);
        ans = 0;
        history.clear();
    }

    void solve(vector<Query> &q) {

        vector<int> res(q.size());
        for(auto [lq, rq, idx] : q) {
            if(rq - lq + 1 > sq) continue;
            for (int i = lq; i <= rq; ++i) {
                add(i, true);
            }
            res[idx] = ans;
            rollback();
        }

        sort(q.begin(), q.end(), [&](auto &lf, auto &ri) {
            if(lf.l / sq == ri.l / sq)
                return lf.r < ri.r;
            return lf.l / sq < ri.l / sq;
        });

        int l = 0, r = -1, block = -1;
        for(auto [lq, rq, idx] : q) {
            if(rq - lq + 1 <= sq) continue;

            if(lq / sq != block) {
                block = lq / sq;
                reset();

                r = min((block + 1) * sq - 1, n - 1);
                l = r + 1;
            }

            while (r < rq) add(++r);

            int old_l = l;
            while (l > lq) add(--l, true);
            res[idx] = ans;
            rollback();
            l = old_l;
        }

        for (int i = 0; i < res.size(); ++i) {
            cout << res[i] << '\n';
        }
    }
};
```

```
    }
};
```

MatrixPow.h

Description: Basic matrix struct for fast exponentiation. Can be used to solve linear recurrences in $O(N^3 \log K)$ time. - When a recurrence depends on i^2 , construct it as a complete square: $(i+1)^2 = i^2 + 2i + 1$. - Fermat's Little Theorem (FLT): $a^{p-1} \equiv 1 \pmod p$. When working with powers, take exponents modulo $(p-1)$. - For multiplication recurrences, you can take the ln of both sides to linearize it!

Time: $\mathcal{O}(N^3 \log K)$

```

e1d730, 28 lines

struct Matrix {
    vector<vector<ll>>m;
    int _n;
    Matrix(int n, int v = 0) : _n(n) {
        m.assign(_n,vector<ll>(_n,v));
    }
    Matrix operator*(Matrix&other) const{
        Matrix ans(_n);
        for (ll i = 0; i < _n; i++)
            for (ll j = 0; j < _n; j++)
                for (ll k = 0; k < _n; k++)
                    ans.m[i][j]+=m[i][k]*other.m[k][j],
                    ans.m[i][j]%=MOD;
        return ans;
    }

    Matrix operator^(ll pw) const{
        Matrix result(_n);
        for(int i = 0; i < _n; i++) result.m[i][i]=1;
        Matrix trans = *this;
        while(pw){
            if(pw&1)result=result*trans;
            trans=trans*trans;
            pw/=2;
        }
        return result;
    }
};
```

XorBasis.h

Description: Linear basis over GF(2) (XOR basis). Set lg to the number of bits needed (e.g. 30 for 10^9 , 60 for 10^{18}). Classical uses: - Max/Min XOR of any subset: maximize(0) / minimize(0). - Check if x is XOR-representable: can(x). - Count distinct XOR values: 2^{size} (ignoring zero if not in span). - k -th smallest XOR value (1-indexed): kth(k), returns -1 if $k > 2^{\text{size}}$. - AND/XOR the whole basis by a mask: AND(x) / XOR(x). - NOTE: xorSet tracks which indices form each basis vector; commented out by default.

Time: $\mathcal{O}(L)$ per insert/query where $L = \lg$.

```

c907ff, 107 lines

// init
const int lg = 0;

struct XorBasis {
    int size;
    set<int> build[lg];
    array<int, lg> basis;

    void init() {
        size = 0;
        for (int i = lg - 1; i >= 0; --i)
            basis[i] = 0, build[i].clear();
    }

    bool insert(int x) {
        for (int i = lg - 1; i >= 0; --i) {
            if ( ( (x >> i) & 1 ) == 0 ) continue;
            if (not basis[i]) return true;
```

```

        x = (x ^ basis[i]);
    }
    return false;
}

// careful gives time limit if used a lot
void xorSet(set<int>& me, set<int>& with) {
    for (int x : with) {
        if ( me.count(x) ) me.erase(x);
        else me.insert(x);
    }
}

void insert(int x, int index) {
    if ( not insert(x) ) return;
    set<int> st = { index };
    for (int i = lg - 1; i >= 0; --i) {
        if ( ( (x >> i) & 1 ) == 0 ) continue;
        if (not basis[i]) {
            basis[i] = x, ++size;
            build[i] = st;
            return;
        }
        if (compare[i] < myValue) {
            swap(comare[i], myValue);
            swap(bas[i], x);
        }
    }
    x = (x ^ basis[i]);
    xorSet(st, build[i]);
}

bool can(int x) {
    for (int i = lg - 1; i >= 0; --i)
        if ( basis[i] and ( (x >> i) & 1 ) )
            x = (x ^ basis[i]);
    return x == 0;
}

// kth one indexed
// gets kth ignoring repeated ways to get the number
// careful if the problem ask for kth considering repeating or not
int kth(int k) {
    int x = 0;
    for (int i = lg - 1, c = size; i >= 0; --i) {
        if (not basis[i]) continue;
        --c;
        if ( ( (x >> i) & 1 ) ) {
            if ( (1LL << c) >= k ) x = (x ^ basis[i]);
            else k = k - (1LL << c);
        } else if ( k > (1LL << c) ) {
            x = (x ^ basis[i]), k = k - (1LL << c);
        }
    }
    // ignore if needed
    if (k > 1) return -1;
    return x;
}

int minimize(int x) {
    for (int i = lg - 1; i >= 0; --i)
        x = min( x, (x ^ basis[i]) );
    return x;
}

int maximized(int x) {
    for (int i = lg - 1; i >= 0; --i)
        x = max( x, (x ^ basis[i]) );
```

```

        return x;
    }

    void AND(int x) {
        size = 0;
        vector<int> update;
        for (int i = lg - 1; i >= 0; --i) {
            basis[i] = (basis[i] & x);
            if (basis[i]) update.push_back(basis[i]);
            basis[i] = 0;
        }
        for (int& value : update) insert(value);
    }

    // you should shift and make leading one and check for that one
    // when you want the actual basis remove the shift - insert again and get your actual basis
    void XOR(int x) {
        for (int i = lg - 1; i >= 0; --i)
            if (basis[i] & 1) basis[i] ^= x;
    }
};
```

Numerical (4)

4.1 Polynomials and recurrences

Polynomial.h

```

c9b7b0, 17 lines

struct Poly {
    vector<double> a;
    double operator()(double x) const {
        double val = 0;
        for (int i = sz(a); i--;) (val *= x) += a[i];
        return val;
    }
    void diff() {
        rep(i,1,sz(a)) a[i-1] = i*a[i];
        a.pop_back();
    }
    void divroot(double x0) {
        double b = a.back(), c; a.back() = 0;
        for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b, b=c;
        a.pop_back();
    }
};
```

4.2 Matrices

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.

Time: $\mathcal{O}(N^3)$

```

bd5cec, 15 lines

double det(vector<vector<double>>& a) {
    int n = sz(a); double res = 1;
    rep(i,0,n) {
        int b = i;
        rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
        if (i != b) swap(a[i], a[b]), res *= -1;
        res *= a[i][i];
        if (res == 0) return 0;
        rep(j,i+1,n) {
            double v = a[j][i] / a[i][i];
            if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
        }
    }
    return res;
}
```

4.3 Fourier transforms

FastFourierTransform.h

Description: $\text{fft}(a)$ computes $\hat{f}(k) = \sum x a[x] \exp(2\pi i \cdot kx/N)$ for all k . N must be a power of 2. Modified to multiply two integer polynomials modulo a global mod. $\text{multiply}(a, b)$ computes $c[x] = \sum a[i]b[x-i] \pmod{mod}$. $\text{poly_pow}(\text{poly}, p)$ computes the polynomial to the power of p modulo mod. Rounding is safe if $(\sum A_i B_i) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16} ; higher for random inputs). Otherwise, use NTT/FFTMod.

Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ (~ 1 s for $N = 2^{22}$)

<pre>const long double PI = acos(-1); typedef complex<double> C; #define rep(i,a,b) for(int i = a; i < b; i++) void fft(vector<C>& a) { int n = a.size(), L = 31 - __builtin_clz(n); static vector<complex<long double>> R(2, 1); static vector<C> rt(2, 1); // (^ 10% faster if double) for (static int k = 2; k < n; k *= 2) { R.resize(n); rt.resize(n); auto x = polar(1.0L, acos(-1.0L) / k); rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2]; } vector<ll> rev(n); rep(i,0,n) rev[i] = (rev[i / 2] (i & 1) << L) / 2; rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]); for (int k = 1; k < n; k *= 2) for (int i = 0; i < n; i += 2 * k) rep(j,0,k) { C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled) a[i + j + k] = a[i + j] - z; a[i + j] += z; } vector<ll> multiply(const vector<ll> &a, const vector<ll> &b) { if (a.empty() b.empty()) return {}; vector<ll> res(a.size() + b.size() - 1); int L = 32 - __builtin_clz(res.size()), n = 1 << L; vector<C> in(n), out(n); copy(a.begin(),a.end(), begin(in)); rep(i,0,b.size()) in[i].imag(b[i]); fft(in); for (C& x : in) x *= x; rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]); fft(out); rep(i,0,res.size()) res[i] = (mod+(ll)round(imag(out[i]) / (4 * n))) % mod; return res; } vector<ll> poly_pow(vector<ll> poly, int p) { vector<ll> ans{1}; while (p) { if(p&1) ans = multiply(ans, poly); poly = multiply(poly, poly); p >>= 1; } return ans; } }</pre>	e1aa82, 46 lines
---	------------------

<pre>if (a.empty() b.empty()) return {}; vl res(sz(a) + sz(b) - 1); int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M)); vector<C> L(n), R(n), outs(n), outl(n); rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut); rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut); fft(L), fft(R); rep(i,0,n) { int j = -i & (n - 1); outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n); outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / li; } fft(outl), fft(outs); rep(i,0,sz(res)) { ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5); ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5); res[i] = ((av % M * cut + bv) % M * cut + cv) % M; } return res; }</pre>	aa30b7, 46 lines
---	------------------

NumberTheoreticTransform.h

Description: Number Theoretic Transform (NTT) for multiplying two polynomials of size K in $\mathcal{O}(K \log K)$. The modulo P must be NTT-friendly. You can check this by computing $\text{_builtin_ctzll}(P - 1)$. For example, for $P = 998244353$, it prints 23, meaning it supports array sizes up to 2^{23} . As a rule of thumb, if $\text{_builtin_ctzll}(P - 1) \geq 18$, it supports array sizes up to 2^{18} , which is safe for almost all problems with $N = 10^5$. Common NTT mods and primitive roots: 998244353 (3), 1004535809 (3), 469762049 (3), 167772161 (3), 754974721 (11). The current implementation uses primitive root 3 for modulo 998244353.

Time: $\mathcal{O}(K \log K)$

<pre>void ntt(vector<ll>& a, bool invert) { int n = a.size(); for (int i = 1, j = 0; i < n; i++) { int bit = n >> 1; for (; j & bit; bit >>= 1) j ^= bit; j ^= bit; if (i < j) swap(a[i], a[j]); } for (int len = 2; len <= n; len <= 1) { // 3 is the primitive root of 998244353 ll wlen = power(3, (mod - 1) / len, mod); if (invert) wlen = power(wlen, mod - 2, mod); for (int i = 0; i < n; i += len) { ll w = 1; for (int j = 0; j < len / 2; j++) { ll u = a[i + j], v = (a[i + j + len / 2] * w) % mod; a[i + j] = (u + v < mod ? u + v : u + v - mod); a[i + j + len / 2] = (u - v >= 0 ? u - v : u - v + mod); w = (w * wlen) % mod; } } } if (invert) { ll n_inv = power(n, mod - 2, mod); for (ll& x : a) x = (x * n_inv) % mod; } }</pre>	
---	--

<pre>vector<ll> multiply(vector<ll> const& a, vector<ll> const& b) { if (a.empty() b.empty()) return {}; vector<ll> fa(a.begin(), a.end()), fb(b.begin(), b.end()); int n = 1; while (n < a.size() + b.size()) n <= 1; fa.resize(n); fb.resize(n);</pre>	
---	--

<pre>ntt(fa, false); ntt(fb, false); for (int i = 0; i < n; i++) { fa[i] = (fa[i] * fb[i]) % mod; } ntt(fa, true); vector<ll> res(a.size() + b.size() - 1); for (int i = 0; i < res.size(); i++) res[i] = fa[i]; return res; }</pre>	
---	--

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x\oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two.

Time: $\mathcal{O}(N \log N)$

<pre>void FST(vi& a, bool inv) { for (int n = sz(a), step = 1; step < n; step *= 2) { for (int i = 0; i < n; i += 2 * step) rep(j,i,step) { int &u = a[j], &v = a[j + step]; tie(u, v) = inv ? pii(v - u, u) : pii(v, u + v); // AND inv ? pii(v, u - v) : pii(u + v, u); // OR pii(u + v, u - v); // XOR } } if (inv) for (int& x : a) x /= sz(a); // XOR only } vi conv(vi a, vi b) { FST(a, 0); FST(b, 0); rep(i,0,sz(a)) a[i] *= b[i]; FST(a, 1); return a; }</pre>	464cf3, 16 lines
---	------------------

Number theory (5)

5.1 Modular arithmetic

ModularArithmetic.h

Description: Operators for modular arithmetic. You need to set mod to some number first and then you can use the structure.

"euclid.h"

<pre>const ll mod = 17; // change to something else struct Mod { ll x; Mod(ll y) : Mod(y%mod+mod,0){} Mod(ll y,int) : x(y<mod?y:y-mod){} Mod operator+(Mod b) { return {x + b.x,0}; } Mod operator-(Mod b) { return {x - b.x + mod,0}; } Mod operator*(Mod b) { return {x * b.x % mod,0}; } Mod operator/(Mod b) { return *this * invert(b); } Mod invert(Mod a) { ll x, y, g = euclid(a.x, mod, x, y); assert(g == 1); return x; } Mod operator^(ll e) { if (!e) return 1; Mod r = *this ^ (e / 2); r = r * r; return e&1 ? *this * r : r; } };</pre>	ealac7, 19 lines
---	------------------

ModInverse.h

Description: Pre-computation of modular inverses. Assumes $\text{LIM} \leq \text{mod}$ and that mod is a prime.

```
6f684f, 3 lines
```

```
const ll mod = 1000000007, LIM = 200000;
ll* inv = new ll[LIM] - 1; inv[1] = 1;
rep(i,2,LIM) inv[i] = mod - (mod / i) * inv[mod % i] % mod;
```

ModPow.h

```
b83e45, 8 lines
```

```
const ll mod = 1000000007; // faster if const

ll modpow(ll b, ll e) {
    ll ans = 1;
    for (; e; b = b * b % mod, e /= 2)
        if (e & 1) ans = ans * b % mod;
    return ans;
}
```

ModLog.h

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. $\text{modLog}(a,1,m)$ can be used to calculate the order of a .

Time: $\mathcal{O}(\sqrt{m})$

```
c040b8, 11 lines
```

```
ll modLog(ll a, ll b, ll m) {
    ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
    unordered_map<ll, ll> A;
    while (j <= n && (e = f = e * a % m) != b % m)
        A[e * b % m] = j++;
    if (e == b % m) return j;
    if (__gcd(m, e) == __gcd(m, b))
        rep(i,2,n+2) if (A.count(e = e * f % m))
            return n * i - A[e];
    return -1;
}
```

ModSum.h

Description: Sums of mod'ed arithmetic progressions.

$\text{modsum}(\text{to}, c, k, m) = \sum_{i=0}^{\text{to}-1} (ki + c) \% m$. divsum is similar but for floored division.

Time: $\log(m)$, with a large constant.

```
5c5bc5, 16 lines
```

```
typedef unsigned long long ull;
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }

ull divsum(ull to, ull c, ull k, ull m) {
    ull res = k / m * sumsq(to) + c / m * to;
    k %= m; c %= m;
    if (!k) return res;
    ull to2 = (to * k + c) / m;
    return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
}

ll modsum(ull to, ll c, ll k, ll m) {
    c = ((c % m) + m) % m;
    k = ((k % m) + m) % m;
    return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

ModMulLL.h

Description: Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$.

Time: $\mathcal{O}(1)$ for modmul , $\mathcal{O}(\log b)$ for modpow

```
bbbd8f, 11 lines
```

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) {
    ll ret = a * b - M * ull(1.L / M * a * b);
    return ret + M * (ret < 0) - M * (ret >= (ll)M);
}

ull modpow(ull b, ull e, ull mod) {
    ull ans = 1;
```

```
for (; e; b = modmul(b, b, mod), e /= 2)
    if (e & 1) ans = modmul(ans, b, mod);
return ans;
}
```

ModSqrt.h

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution).

Time: $\mathcal{O}(\log^2 p)$ worst case, $\mathcal{O}(\log p)$ for most p

```
"ModPow.h"19a793, 24 lines
```

```
ll sqrt(ll a, ll p) {
    a %= p; if (a < 0) a += p;
    if (a == 0) return 0;
    assert(modpow(a, (p-1)/2, p) == 1); // else no solution
    if (p % 4 == 3) return modpow(a, (p+1)/4, p);
    // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
    ll s = p - 1, n = 2;
    int r = 0, m;
    while (s % 2 == 0)
        ++r, s /= 2;
    while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
    ll x = modpow(a, (s + 1) / 2, p);
    ll b = modpow(a, s, p), g = modpow(n, s, p);
    for (; r = m) {
        ll t = b;
        for (m = 0; m < r && t != 1; ++m)
            t = t * t % p;
        if (m == 0) return x;
        ll gs = modpow(g, 1LL << (r - m - 1), p);
        g = gs * gs % p;
        x = x * gs % p;
        b = b * g % p;
    }
}
```

5.2 Primality

FastEratosthenes.h

Description: Prime sieve for generating all primes smaller than LIM.

Time: $\text{LIM} = 1e9 \approx 1.5s$

```
6b2912, 20 lines
```

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
    const int S = (int)round(sqrt(LIM)), R = LIM / 2;
    vi pr = {2}, sieve(S+1); pr.reserve((int)(LIM/log(LIM)*1.1));
    vector<pii> cp;
    for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
        cp.push_back({i, i * i / 2});
        for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
    }
    for (int L = 1; L <= R; L += S) {
        array<bool, S> block{};
        for (auto &[p, idx] : cp)
            for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
        rep(i,0,min(S, R - L))
            if (!block[i]) pr.push_back((L + i) * 2 + 1);
    }
    for (int i : pr) isPrime[i] = 1;
    return pr;
}
```

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.

Time: 7 times the complexity of $a^b \bmod c$.

```
"ModMulLL.h"60dcd1, 12 lines
```

```
bool isPrime(ull n) {
```

```
if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
s = __builtin_ctzll(n-1), d = n >> s;
for (ull a : A) { // ^ count trailing zeroes
    ull p = modpow(a%n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
        p = modmul(p, p, n);
    if (p != n-1 && i != s) return 0;
}
return 1;
}
```

Factor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).

Time: $\mathcal{O}(n^{1/4})$, less for numbers with small factors.

```
"ModMulLL.h", "MillerRabin.h"d8d98d, 18 lines
```

```
ull pollard(ull n) {
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    auto f = [&](ull x) { return modmul(x, x, n) + i; };
    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y) x = ++i, y = f(x);
        if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
        x = f(x), y = f(f(y));
    }
    return __gcd(prd, n);
}

vector<ull> factor(ull n) {
    if (n == 1) return {};
    if (isPrime(n)) return {n};
    ull x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
    return l;
}
```

5.3 Divisibility

euclid.h

Description: Finds two integers x and y , such that $ax + by = \gcd(a, b)$. If you just need $\gcd$, use the built in $_gcd$ instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

```
33ba8f, 5 lines
```

```
ll euclid(ll a, ll b, ll &x, ll &y) {
    if (!b) return x = 1, y = 0, a;
    ll d = euclid(b, a % b, y, x);
    return y -= a/b * x, d;
}
```

CRT.h

Description: Chinese Remainder Theorem.

$\text{crt}(a, m, b, n)$ computes x such that $x \equiv a \pmod m, x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \leq x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.

Time: $\log(n)$

```
"euclid.h"04d93a, 7 lines
```

```
ll crt(ll a, ll m, ll b, ll n) {
    if (n > m) swap(a, b), swap(m, n);
    ll x, y, g = euclid(m, n, x, y);
    assert((a - b) % g == 0); // else no solution
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m*n/g : x;
}
```

5.3.1 Bézout’s identity

For $a \neq 0, b \neq 0$, then $d = \gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h

Description: *Euler’s ϕ function* is defined as $\phi(n) := \#$ of positive integers $\leq n$ that are coprime with n . $\phi(1) = 1, p$ prime $\Rightarrow \phi(p^k) = (p - 1)p^{k-1}, m, n$ coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1}p_2^{k_2}...p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1-1}...(p_r - 1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n}(1 - 1/p)$. $\sum_{d|n} \phi(d) = n, \sum_{1 \leq k \leq n, \gcd(k,n)=1} k = n\phi(n)/2, n > 1$
Euler’s thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.
Fermat’s little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod p \forall a$.

```
const int LIM = 5000000;
int phi[LIM];

void calculatePhi() {
    rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
    for (int i = 3; i < LIM; i += 2) if(phi[i] == i)
        for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}
```

5.4 Fractions

ContinuedFractions.h

Description: Given N and a real number $x \geq 0$, finds the closest rational approximation p/q with $p, q \leq N$. It will obey $|p/q - x| \leq 1/qN$. For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. (p_k/q_k) alternates between $> x$ and $< x$.) If x is rational, y eventually becomes ∞ ; if x is the root of a degree 2 polynomial the a ’s eventually become cyclic.
Time: $\mathcal{O}(\log N)$

```
typedef double d; // for N ~ 1e7; long double for N ~ 1e9
pair<ll, ll> approximate(d x, ll N) {
    ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX; d y = x;
    for (;;) {
        ll lim = min(P ? (N-LP) / P : inf, Q ? (N-LQ) / Q : inf),
            a = (ll)floor(y), b = min(a, lim),
            NP = b*P + LP, NQ = b*Q + LQ;
        if (a > b) {
            // If b > a/2, we have a semi-convergent that gives us a
            // better approximation; if b = a/2, we *may* have one.
            // Return {P, Q} here for a more canonical approximation.
            return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q)) ?
                make_pair(NP, NQ) : make_pair(P, Q);
        }
        if (abs(y = 1/(y - (d)a)) > 3*N) {
            return {NP, NQ};
        }
        LP = P; P = NP;
        LQ = Q; Q = NQ;
    }
}
```

FracBinarySearch.h

Description: Given f and N , finds the smallest fraction $p/q \in [0, 1]$ such that $f(p/q)$ is true, and $p, q \leq N$. You may want to throw an exception from f if it finds an exact solution, in which case N can be removed.

```
Usage: fracBS([](Frac f) { return f.p>=3*f.q; }, 10); // {1,3}
Time: O(log(N))

27ab3e, 25 lines

struct Frac { ll p, q; };

template<class F>
Frac fracBS(F f, ll N) {
    bool dir = 1, A = 1, B = 1;
    Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
    if (f(lo)) return lo;
    assert(f(hi));
    while (A || B) {
        ll adv = 0, step = 1; // move hi if dir, else lo
        for (int si = 0; step; (step *= 2) >= si) {
            adv += step;
            Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
            if (abs(mid.p) > N || mid.q > N || dir == !f(mid)) {
                adv -= step; si = 2;
            }
        }
        hi.p += lo.p * adv;
        hi.q += lo.q * adv;
        dir = !dir;
        swap(lo, hi);
        A = B; B = !!adv;
    }
    return dir ? hi : lo;
}
```

5.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2),$$

with $m > n > 0, k > 0, m \perp n$, and either m or n even.

5.6 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a , except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.7 Estimates

$$\sum_{d|n} d = O(n \log \log n).$$

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

5.8 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d)g(n/d)$$

Other useful formulas/forms:

$$\sum_{d|n} \mu(d) = [n = 1] \text{ (very useful)}$$

$$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n)g(d)$$

$$g(n) = \sum_{1 \leq m \leq n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m)g(\lfloor \frac{n}{m} \rfloor)$$

Combinatorial (6)

combinatorics.h

Description: Template for all combinatorics.

```
6e8dd6, 68 lines

namespace combinatorics
{
    ll MOD;
    vector<ll> fac, inv, finv;
    ll nCr(ll x, ll y)
    {
        if (x < 0 || y > x || y < 0) return 0;
        return (fac[x] * finv[y] % MOD * finv[x-y] % MOD);
    }
    ll nPr(ll x, ll y)
    {
        if (x < 0 || y > x || y < 0) return 0;
        return fac[x] * finv[x-y] % MOD;
    }
    ll power(ll b, ll n)
    {
        b %= MOD;
        if (b < 0) b += MOD;
        ll s = 1;
        while (n)
        {
            if (n % 2 == 1) s = (ll)((__int128)s * b % MOD);
            b = (ll)((__int128)b * b % MOD);
            n /= 2;
        }
        return s;
    }
    void init(int n, ll mod)
    {
        fac.resize(n+1);
        inv.resize(n+1);
        finv.resize(n+1);
        MOD = mod;
        fac[0] = inv[0] = inv[1] = finv[0] = finv[1] = 1;
        for (ll i = 1; i <= n; ++i) fac[i] = fac[i-1] * i % MOD;
        for (ll i = 2; i <= n; ++i) inv[i] = MOD - MOD / i * inv[MOD % i] % MOD;
        for (ll i = 2; i <= n; ++i) finv[i] = finv[i-1] * inv[i] % MOD;
    }
    ll mul(ll a, ll b)
    {
        return ((a % MOD) * (b % MOD)) % MOD;
    }
    ll add(ll a, ll b)
    {
        return ((a % MOD) + (b % MOD)) % MOD;
    }
    ll sub(ll a, ll b)
    {
        return (((a - b) % MOD) + MOD) % MOD;
    }
    ll divide(ll a, ll b)
    {
        return mul(a, power(b, MOD - 2));
    }
    ll Inv(ll x)
    {

```

```
        return power(x,MOD-2);
    }
    ll catalan(int n)
    {
        return (nCr(2*n,n)*Inv(n+1))%MOD;
    }
    ll StarsAndPars(ll n,ll k)
    {
        return nCr(n+k-1,k-1);
    }
};
using namespace combinatorics;
```

6.1 Permutations

6.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

6.1.2 Cycles

Let $g_S(n)$ be the number of n -permutations whose cycle lengths all belong to the set S . Then

$$\sum_{n=0}^\infty g_S(n) \frac{x^n}{n!} = \exp\left(\sum_{n \in S} \frac{x^n}{n}\right)$$

6.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1)+D(n-2)) = nD(n-1)+(-1)^n = \left\lfloor \frac{n!}{e} \right\rfloor$$

6.2 General purpose numbers

6.2.1 Catalan numbers

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$
$$C_0 = 1, \ C_{n+1} = \frac{2(2n+1)}{n+2} C_n, \ C_{n+1} = \sum C_i C_{n-i}$$
$$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$$

Graph (7)

7.1 Fundamentals

TopoSort.h

Description: Topological sorting of a directed graph using DFS. Produces an ordering of all vertices such that for every directed edge $u \rightarrow v$, u appears before v in the ordering, provided the graph is acyclic. Vertices are added to the result after all their outgoing neighbors have been processed, then the resulting list is reversed. This implementation does not explicitly detect cycles. For graphs containing cycles, the returned ordering is not guaranteed to be valid.

Memory: $\mathcal{O}(|V|)$

Time: $\mathcal{O}(|V| + |E|)$

```
int n; // number of vertices
vector<vector<int>> adj; // adjacency list of graph
vector<bool> visited;
vector<int> ans;

void dfs(int v) {
    visited[v] = true;
    for (int u : adj[v]) {
        if (!visited[u]) {
            dfs(u);
        }
    }
    ans.push_back(v);
}

void topological_sort() {
    visited.assign(n, false);
    ans.clear();
    for (int i = 0; i < n; ++i) {
        if (!visited[i]) {
            dfs(i);
        }
    }
    reverse(ans.begin(), ans.end());
}
```

FloydWarshallAlgorithm.h

Description: All-Pairs Shortest Paths using the Floyd-Warshall algorithm. Computes the shortest distance between every pair of vertices by allowing each vertex to be used as an intermediate vertex. The distance matrix d should initially contain edge weights, with $d[i][i] = 0$ and $d[i][j] = \text{INF}$ when there is no direct path between i and j . Supports negative edge weights, but the presence of negative cycles makes the shortest path distances undefined.

Memory: $\mathcal{O}(|V|^2)$

Time: $\mathcal{O}(|V|^3)$

```
// All Pairs Shortest Path
void floydWarshall(vector<vector<ll>>& d) {
    int n = d.size();
    for (int k = 0; k < n; ++k) {
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < n; ++j) {
                if (d[i][k] != INF && d[k][j] != INF) {
                    d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
                }
            }
        }
    }
}
```

Dijkstra.h

Description: Dijkstra's algorithm for finding the shortest paths from a single source vertex to all other vertices in a weighted graph. Uses a min-heap priority queue to repeatedly process the vertex with the smallest known distance and relax its outgoing edges. Requires all edge weights to be non-negative. Unreachable vertices retain a distance of INF .

Memory: $\mathcal{O}(|V| + |E|)$

Time: $\mathcal{O}((|V| + |E|)\log|V|)$

```
#define long long ll;
const ll INF = 1e18;
vector<vector<pair<int, ll>>> adj;

vector<ll> dijkstra(int n, int src) {
    vector<ll> dist(n, INF); // distance array
    dist[src] = 0;

    // Min-heap: {distance, node}
    priority_queue<pair<ll, int>, vector<pair<ll, int>>,
        greater<>> pq;
    pq.push({0, src});

    while (!pq.empty()) {
        auto [currentDist, u] = pq.top();
        pq.pop();

        // Skip if this is not the best distance anymore
        if (currentDist > dist[u]) continue;

        // Traverse neighbors
        for (auto &[v, weight] : adj[u]) {
            // Relaxation step
            if (dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                pq.push({dist[v], v});
            }
        }
    }

    return dist;
}
```

Kruskal.h

Description: Minimum Spanning Tree using Kruskal's algorithm. Sorts all edges by increasing weight and greedily adds an edge whenever its endpoints belong to different connected components. Uses a Disjoint Set Union structure with path compression and union by rank to efficiently detect and merge connected components. If the graph is connected, the result contains a minimum spanning tree. For a disconnected graph, the result is a minimum spanning forest.

Memory: $\mathcal{O}(|V| + |E|)$

Time: $\mathcal{O}(|E|\log|E|)$

```
vector<int> parent, rank;

// Initialize a set with a single element (self parent)
void make_set(int v) {
    parent[v] = v; // each node is its own parent initially
    rank[v] = 0; // rank (tree height approximation)
    starts at 0
}

// Find the representative (root) of the set with path
// compression
int find_set(int v) {
    if (v == parent[v])
        return v;
    // Path compression: flatten the tree for faster future
    // queries
    return parent[v] = find_set(parent[v]);
}
```

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n + 1$ leaves (0 or 2 children).
- ordered trees with $n + 1$ vertices.
- ways a convex polygon with $n + 2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

```
// Union two sets by rank (attach smaller tree under larger tree)
void union_sets(int a, int b) {
    a = find_set(a); // find root of a
    b = find_set(b); // find root of b

    if (a != b) { // only merge if they are in different sets
        // Ensure that 'a' has higher rank
        if (rank[a] < rank[b])
            swap(a, b);

        parent[b] = a; // attach tree b under root a

        // If both have same rank, increase rank of new root
        if (rank[a] == rank[b])
            rank[a]++;
    }
}

// Edge structure for graph
struct Edge {
    int u, v, weight;

    // Comparator to sort edges by weight (ascending)
    bool operator<(Edge const& other) {
        return weight < other.weight;
    }
};

int n; // number of vertices
vector<Edge> edges; // list of all edges

int cost = 0; // total cost of MST
vector<Edge> result; // edges included in MST

parent.resize(n);
rank.resize(n);
for (int i = 0; i < n; i++)
    make_set(i);

sort(edges.begin(), edges.end());

// Iterate over edges in increasing order of weight
for (Edge e : edges) {
    // If adding this edge doesn't form a cycle
    if (find_set(e.u) != find_set(e.v)) {
        cost += e.weight; // include edge weight in MST cost
        result.push_back(e); // store edge in MST
        union_sets(e.u, e.v); // merge the components
    }
}

// Description: Strongly Connected Components using Kosaraju's algorithm.
// Decomposes a directed graph into maximal sets of vertices where every vertex
// is reachable from every other vertex in the same component. Performs
// two depth-first search passes: the first determines vertices' finishing order,
// while the second runs on the reversed graph to identify the strongly connected
// components. Also constructs the condensation graph, where each SCC is represented
// by its root vertex and edges between different SCCs are preserved.
// Memory: O(|V| + |E|)
// Time: O(|V| + |E|)
```

5a2d60, 59 lines

```
vector<bool> visited; // keeps track of which vertices are already visited
```

```
// runs depth first search starting at vertex v.
// each visited vertex is appended to the output vector when dfs leaves it.
void dfs(int v, vector<vector<int>>> const& adj, vector<int> & output) {
    visited[v] = true;
    for (auto u : adj[v])
        if (!visited[u])
            dfs(u, adj, output);
    output.push_back(v);
}

// input: adj — adjacency list of G
// output: components — the strongly connected components in G
// output: adj_cond — adjacency list of G^SCC (by root vertices)
void strongly_connected_components(vector<vector<int>>> const& adj,
                                   vector<vector<int>>> & components,
                                   vector<vector<int>>> & adj_cond) {
    int n = adj.size();
    components.clear(), adj_cond.clear();

    vector<int> order; // will be a sorted list of G's vertices by exit time

    visited.assign(n, false);

    // first series of depth first searches
    for (int i = 0; i < n; i++)
        if (!visited[i])
            dfs(i, adj, order);

    // create adjacency list of G^T
    vector<vector<int>>> adj_rev(n);
    for (int v = 0; v < n; v++)
        for (int u : adj[v])
            adj_rev[u].push_back(v);

    visited.assign(n, false);
    reverse(order.begin(), order.end());

    vector<int> roots(n, 0); // gives the root vertex of a vertex's SCC

    // second series of depth first searches
    for (auto v : order)
        if (!visited[v]) {
            std::vector<int> component;
            dfs(v, adj_rev, component);
            components.push_back(component);
            int root = *min_element(begin(component), end(component));
            for (auto u : component)
                roots[u] = root;
        }

    // add edges to condensation graph
    adj_cond.assign(n, {});
    for (int v = 0; v < n; v++)
        for (auto u : adj[v])
            if (roots[v] != roots[u])
                adj_cond[roots[v]].push_back(roots[u]);
}
```

BFS.h

Description: Breadth-First Search (BFS) on an unweighted graph. Computes the shortest distance from a source vertex to every reachable vertex by exploring the graph level by level. Also stores the parent of each vertex, allowing the shortest path from the source to any reachable vertex to be reconstructed. Vertices that are unreachable from the source retain a distance of -1. Works for both directed and undirected graphs depending on the adjacency list provided.

Memory: $O(|V|)$

Time: $O(|V| + |E|)$

545157, 56 lines

```
vector<int> bfs(const vector<vector<int>>& adj, int src,
               vector<int>& parent) {
    int n = adj.size();

    vector<int> dist(n, -1);
    parent.assign(n, -1);

    if (src < 0 || src >= n)
        return dist;

    queue<int> q;

    dist[src] = 0;
    q.push(src);

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            // Already visited
            if (dist[v] != -1)
                continue;

            dist[v] = dist[u] + 1;
            parent[v] = u;

            q.push(v);
        }

        return dist;
    }
}

vector<int> get_path(int src, int target,
                    const vector<int>& parent,
                    const vector<int>& dist) {
    if (target < 0 || target >= (int)parent.size())
        return {};

    if (dist[target] == -1)
        return {};

    vector<int> path;

    for (int v = target; v != -1; v = parent[v])
        path.push_back(v);

    reverse(path.begin(), path.end());

    if (path.empty() || path.front() != src)
        return {};

    return path;
}
```

MultiSourceBFS.h

Description: Multi-source BFS on a 2D grid. Computes the shortest Manhattan distance from every cell to its nearest source cell by initializing the BFS queue with all sources simultaneously. Each move is allowed in the four cardinal directions and has unit cost. Unreachable cells retain a distance of INF.

Memory: $\mathcal{O}(n * m)$

Time: $\mathcal{O}(|V| + |E|) = \mathcal{O}(n * m)$

293f07, 65 lines

```
int dx[] = {1, -1, 0, 0};
int dy[] = {0, 0, 1, -1};

vector<vector<int>> multi_source_bfs_grid(vector<vector<int>>&
    grid,
    vector<pair<int,int>>
        sources) {

    int n = grid.size();
    int m = grid[0].size();

    vector<vector<int>> dist(n, vector<int>(m, INF));
    queue<pair<int,int>> q;

    // initialize sources
    for (auto [x, y] : sources) {
        dist[x][y] = 0;
        q.push({x, y});
    }

    while (!q.empty()) {
        auto [x, y] = q.front();
        q.pop();

        for (int d = 0; d < 4; d++) {
            int nx = x + dx[d];
            int ny = y + dy[d];

            if (nx >= 0 && nx < n && ny >= 0 && ny < m) {
                if (dist[nx][ny] > dist[x][y] + 1) {
                    dist[nx][ny] = dist[x][y] + 1;
                    q.push({nx, ny});
                }
            }
        }
    }

    return dist;
}

const int INF = 1e9;

vector<int> multi_source_bfs(int n, vector<vector<int>>& adj,
    vector<int>& sources) {
    vector<int> dist(n, INF);
    queue<int> q;

    // initialize all sources
    for (int src : sources) {
        dist[src] = 0;
        q.push(src);
    }

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (dist[v] > dist[u] + 1) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
}
```

DFS.h

Description: Depth-First Search (DFS) with vertex entry and exit times. Assigns each vertex an entry time when DFS first visits it and an exit time after all of its reachable descendants have been processed. Uses three colors to represent the DFS state: unvisited, currently active, and fully processed. The entry and exit times can be used to determine ancestor relationships and are useful in algorithms such as cycle detection and topological sorting. For a graph with multiple connected components, DFS should be started from every unvisited vertex.

Memory: $\mathcal{O}(|V|)$

Time: $\mathcal{O}(|V| + |E|)$

a79d7c, 17 lines

```
vector<vector<int>> adj; // graph represented as an adjacency
    list
int n; // number of vertices

vector<int> color;

vector<int> time_in, time_out;
int dfs_timer = 0;

void dfs(int v) {
    time_in[v] = dfs_timer++;
    color[v] = 1;
    for (int u : adj[v])
        if (color[u] == 0)
            dfs(u);
    color[v] = 2;
    time_out[v] = dfs_timer++;
}
```

7.2 Trees

LCA.h

Description: Lowest Common Ancestor (LCA) using Binary Lifting. Pre-processes a rooted tree to store the 2 to the i-th ancestor of every vertex along with its depth. Supports finding the k-th ancestor of a vertex and finding the lowest common ancestor of any two vertices. The implementation assumes the graph is a tree and uses DFS to build the ancestor table. LOG must be large enough to represent the maximum possible tree depth.

Memory: $\mathcal{O}(|V| \log |V|)$

Time: $\mathcal{O}(|V| \log |V|)$ preprocessing, $\mathcal{O}(\log |V|)$ per query

490afe, 56 lines

```
// int N , LOG;
// int up[N][LOG];
// int depth[N];

vector<vector<int>> adjList;

void dfs(int u, int p, int d){
    depth[u] = d;

    up[u][0] = p;
    for (size_t i = 1; i < LOG; i++)
    {
        if (up[u][i - 1] != -1) {
            up[u][i] = up[ up[u][i - 1] ][i - 1];
        }
    }

    for(auto next : adjList[u]){
        if(next == p) continue;

        dfs(next, u, d + 1);
    }
}
```

```
    }
}

int kth(int u, int p){
    if(depth[u] < p){
        return -1;
    }

    for (size_t i = 0; i < LOG; i++)
    {
        if(p & (1 << i)){
            u = up[u][i];
        }
    }
    return u;
}

int lca(int a, int b){
    if(depth[a] > depth[b]){
        swap(a, b);
    }

    b = kth(b, depth[b] - depth[a]);

    if(a == b) return a;

    for (int i = LOG - 1; i >= 0; i--)
    {
        if(up[a][i] != up[b][i]){
            a = up[a][i];
            b = up[b][i];
        }
    }
    return up[a][0];
}
```

EulerTree.h

Description: Euler Tour / DFS Flattening of a tree. Assigns each vertex a contiguous interval [tin[u], tout[u]] such that the entire subtree of u appears consecutively in the flattened array. tin[u] stores the position where the subtree of u starts, while tout[u] stores the position where the subtree ends. The flattened array can be used to transform subtree queries on a tree into range queries over a one-dimensional array. Requires the input graph to be a tree and a valid root to be chosen before calling DFS.

Memory: $\mathcal{O}(|V|)$

Time: $\mathcal{O}(|V|)$

ba7668, 17 lines

```
// subtree of node u is flat -> [tin[u] : tout[u]]
vector<int> adj[N];
int tin[N], tout[N], flat[N];
int timer = 0;

void dfs(int u, int parent) {
    tin[u] = timer;
    flat[timer] = u;
    timer++;

    for (auto v : adj[u]) {
        if (v == parent) continue;
        dfs(v, u);
    }

    tout[u] = timer - 1;
}
```


7.2.1 Number of Spanning Trees

Create an $N \times N$ matrix `mat`, and for each edge $a \rightarrow b \in G$, do `mat[a][b]--`, `mat[b][b]++` (and `mat[b][a]--`, `mat[a][a]++` if G is undirected). Remove the i th row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column).

7.2.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \dots \geq d_n$ exists iff $d_1 + \dots + d_n$ is even and for every $k = 1 \dots n$,

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k).$$

Geometry (8)

8.1 Geometric primitives

Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

```
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
    typedef Point P;
    T x, y;
    explicit Point(T x=0, T y=0) : x(x), y(y) {}
    bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
    bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
    P operator+(P p) const { return P(x+p.x, y+p.y); }
    P operator-(P p) const { return P(x-p.x, y-p.y); }
    P operator*(T d) const { return P(x*d, y*d); }
    P operator/(T d) const { return P(x/d, y/d); }
    T dot(P p) const { return x*p.x + y*p.y; }
    T cross(P p) const { return x*p.y - y*p.x; }
    T cross(P a, P b) const { return (a-*this).cross(b-*this); }
    T dist2() const { return x*x + y*y; }
    double dist() const { return sqrt((double)dist2()); }
    // angle to x-axis in interval [-pi, pi]
    double angle() const { return atan2(y, x); }
    P unit() const { return *this/dist(); } // makes dist()=1
    P perp() const { return P(-y, x); } // rotates +90 degrees
    P normal() const { return perp().unit(); }
    // returns point rotated 'a' radians ccw around the origin
    P rotate(double a) const {
        return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
    friend ostream& operator<<(ostream& os, P p) {
        return os << "(" << p.x << ", " << p.y << ")"; }
};
```

lineDistance.h

Description: Returns the signed distance between point `p` and the line containing points `a` and `b`. Positive value on left side and negative on right as seen from `a` towards `b`. `a==b` gives nan. `P` is supposed to be `Point<T>` or `Point3D<T>` where `T` is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using `Point3D` will always give a non-negative distance. For `Point3D`, call `.dist` on the result of the cross product.

"Point.h" 53630a, 4 lines

```
template<class P>
double lineDist(P a, P b, P p) {
    return (double) (b-a).cross(p-a) / (b-a).dist();
}
```

SegmentDistance.h

Description: Returns the shortest distance between point `p` and the line segment from point `s` to `e`.
Usage: `Point<double> a, b(2,2), p(1,1);`
`bool onSegment = segDist(a,b,p) < 1e-10;`

SegmentIntersection.h

Description: If a unique intersection point between the line segments going from `s1` to `e1` and from `s2` to `e2` exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if `P` is `Point<ll>` and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.
Usage: `vector<P> inter = segInter(s1,e1,s2,e2);`
`if (sz(inter)==1)`
`cout << "segments intersect at " << inter[0] << endl;`

```
"Point.h", "OnSegment.h" 9d57f2, 13 lines
template<class P> vector<P> segInter(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(d, b),
        oc = a.cross(b, c), od = a.cross(b, d);
    // Checks if intersection is single non-endpoint point.
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        return {(a * ob - b * oa) / (ob - oa)};
    set<P> s;
    if (onSegment(c, d, a)) s.insert(a);
    if (onSegment(c, d, b)) s.insert(b);
    if (onSegment(a, b, c)) s.insert(c);
    if (onSegment(a, b, d)) s.insert(d);
    return {all(s)};
}
```

lineIntersection.h

Description: If a unique intersection point of the lines going through `s1,e1` and `s2,e2` exists `{1, point}` is returned. If no intersection point exists `{0, (0,0)}` is returned and if infinitely many exists `{-1, (0,0)}` is returned. The wrong position will be returned if `P` is `Point<ll>` and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.
Usage: `auto res = lineInter(s1,e1,s2,e2);`
`if (res.first == 1)`
`cout << "intersection point at " << res.second << endl;`

```
"Point.h" a01f81, 8 lines
template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0) // if parallel
        return {(s1.cross(e1, s2) == 0), P(0, 0)};
    auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
```

```
    return {1, (s1 * p + e1 * q) / d};
}
```

sideOf.h

Description: Returns where p is as seen from s towards e . $1/0/-1 \Leftrightarrow$ left/on line/right. If the optional argument `eps` is given 0 is returned if p is within distance `eps` from the line. `P` is supposed to be `Point<T>` where `T` is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.
Usage: `bool left = sideOf(p1,p2,q)==1;`

"Point.h" ca4665, 9 lines

```
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }
```

```
template<class P>
int sideOf(P s, P e, P p, double eps) {
    auto a = (e-s).cross(p-s);
    double l = (e-s).dist()*eps;
    return (a > l) - (a < -l);
}
```

OnSegment.h

Description: Returns true iff `p` lies on the line segment from `s` to `e`. Use `(segDist(s,e,p) <= epsilon)` instead when using `Point<double>`.

"Point.h" c597e8, 3 lines

```
template<class P> bool onSegment(P s, P e, P p) {
    return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

linearTransformation.h

Description: Apply the linear transformation (translation, rotation and scaling) which takes line `p0-p1` to line `q0-q1` to point `r`.

```
"Point.h" 7871e9, 5 lines
typedef Point<double> P;
P linearTransformation(P p0, P p1, P q0, P q1, P r) {
    P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));
    return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();
}
```

Angle.h

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.
Usage: `vector<Angle> v = {w[0], w[0].t360() ...};` // sorted
`int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }`
// sweeps `j` such that `(j-i)` represents the number of positively oriented triangles with vertices at 0 and `i`

0f0602, 35 lines

```
struct Angle {
    int x, y;
    int t;
    Angle(int x, int y, int t=0) : x(x), y(y), t(t) {}
    Angle operator-(Angle b) const { return {x-b.x, y-b.y, t}; }
    int half() const {
        assert(x || y);
        return y < 0 || (y == 0 && x < 0);
    }
    Angle t90() const { return {-y, x, t + (half() && x >= 0)}; }
    Angle t180() const { return {-x, -y, t + half(); }
    Angle t360() const { return {x, y, t + 1}; }
};
bool operator<(Angle a, Angle b) {
    // add a.dist2() and b.dist2() to also compare distances
    return make_tuple(a.t, a.half(), a.y * (ll)b.x) <
        make_tuple(b.t, b.half(), a.x * (ll)b.y);
}
```

```
// Given two points, this calculates the smallest angle between
// them, i.e., the angle that covers the defined line segment.
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
    if (b < a) swap(a, b);
    return (b < a.t180() ?
        make_pair(a, b) : make_pair(b, a.t360()));
}
Angle operator+(Angle a, Angle b) { // point a + vector b
    Angle r(a.x + b.x, a.y + b.y, a.t);
    if (a.t180() < r) r.t--;
    return r.t180() < a ? r.t360() : r;
}
Angle angleDiff(Angle a, Angle b) { // angle b - angle a
    int tu = b.t - a.t; a.t = b.t;
    return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b < a)};
}
```

8.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

"Point.h"	84d6d3, 11 lines
<pre>typedef Point<double> P; bool circleInter(P a,P b,double r1,double r2,pair<P, P>* out) { if (a == b) { assert(r1 != r2); return false; } P vec = b - a; double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2, p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2; if (sum*sum < d2 dif*dif > d2) return false; P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d2); *out = {mid + per, mid - per}; return true; }</pre>	

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

"Point.h"	b0153d, 13 lines
<pre>template<class P> vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) { P d = c2 - c1; double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr; if (d2 == 0 h2 < 0) return {}; vector<pair<P, P>> out; for (double sign : {-1, 1}) { P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2; out.push_back({c1 + v * r1, c2 + v * r2}); } if (h2 == 0) out.pop_back(); return out; }</pre>	

CirclePolygonIntersection.h

Description: Returns the area of the intersection of a circle with a ccw polygon.

Time: $\mathcal{O}(n)$

"../content/geometry/Point.h"	19add1, 19 lines
<pre>typedef Point<double> P; #define arg(p, q) atan2(p.cross(q), p.dot(q)) double circlePoly(P c, double r, vector<P> ps) { auto tri = [&](P p, P q) {</pre>	

<pre>auto r2 = r * r / 2; P d = q - p; auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.dist2(); auto det = a * a - b; if (det <= 0) return arg(p, q) * r2; auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det)); if (t < 0 1 <= s) return arg(p, q) * r2; P u = p + d * s, v = q + d * (t-1); return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2; }; auto sum = 0.0; rep(i,0,sz(ps)) sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c); return sum; }</pre>	
--	--

circumcircle.h

Description:

The circumcirle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.

"Point.h"	6cffc5, 9 lines
<pre>typedef Point<double> P; double ccRadius(P A, P B, P C) { return (B-A).dist()*(C-B).dist()*(A-C).dist()/ abs((B-A).cross(C-A))/2; } P ccCenter(P A, P B, P C) { P b = C-A, c = B-A; return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2; }</pre>	

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points.

Time: expected $\mathcal{O}(n)$

"circumcircle.h"	09dd0a, 17 lines
<pre>pair<P, double> mec(vector<P> ps) { shuffle(all(ps), mt19937(time(0))); P o = ps[0]; double r = 0, EPS = 1 + 1e-8; rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) { o = ps[i], r = 0; rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) { o = (ps[i] + ps[j]) / 2; r = (o - ps[i]).dist(); rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) { o = ccCenter(ps[i], ps[j], ps[k]); r = (o - ps[i]).dist(); } } } return {o, r}; }</pre>	

8.3 Polygons

InsidePolygon.h

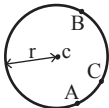
Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.

Usage: vector<P> v = {P{4,4}, P{1,2}, P{2,1}};

bool in = inPolygon(v, P{3, 3}, false);

Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h"	2bf504, 11 lines
<pre>template<class P> bool inPolygon(vector<P> &p, P a, bool strict = true) { int cnt = 0, n = sz(p);</pre>	



<pre>rep(i,0,n) { P q = p[(i + 1) % n]; if (onSegment(p[i], q, a)) return !strict; //or: if (segDist(p[i], q, a) <= eps) return !strict; cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0; } return cnt; }</pre>	
--	--

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

"Point.h"	f12300, 6 lines
<pre>template<class T> T polygonArea2(vector<Point<T>>& v) { T a = v.back().cross(v[0]); rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]); return a; }</pre>	

PolygonCenter.h

Description: Returns the center of mass for a polygon.

Time: $\mathcal{O}(n)$

"Point.h"	9706dc, 9 lines
<pre>typedef Point<double> P; P polygonCenter(const vector<P>& v) { P res(0, 0); double A = 0; for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) { res = res + (v[i] + v[j]) * v[j].cross(v[i]); A += v[j].cross(v[i]); } return res / A / 3; }</pre>	

PolygonCut.h

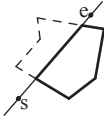
Description:

Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.

Usage: vector<P> p = ...;

p = polygonCut(p, P(0,0), P(1,0));

"Point.h"	d07181, 13 lines
<pre>typedef Point<double> P; vector<P> polygonCut(const vector<P>& poly, P s, P e) { vector<P> res; rep(i,0,sz(poly)) { P cur = poly[i], prev = i ? poly[i-1] : poly.back(); auto a = s.cross(e, cur), b = s.cross(e, prev); if ((a < 0) != (b < 0)) res.push_back(cur + (prev - cur) * (a / (a - b))); if (a < 0) res.push_back(cur); } return res; }</pre>	



ConvexHull.h

Description:

Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.

Time: $\mathcal{O}(n \log n)$

"Point.h"	310954, 13 lines
<pre>typedef Point<ll> P; vector<P> convexHull(vector<P> pts) { if (sz(pts) <= 1) return pts; sort(all(pts)); vector<P> h(sz(pts)+1);</pre>	



```
int s = 0, t = 0;
for (int it = 2; it--; s = --t, reverse(all(pts)))
    for (P p : pts) {
        while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
        h[t++] = p;
    }
return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```

HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).
Time: $\mathcal{O}(n)$

"Point.h"	c571b8, 12 lines
<pre>typedef Point<ll> P; array<P, 2> hullDiameter(vector<P> S) { int n = sz(S), j = n < 2 ? 0 : 1; pair<ll, array<P, 2>> res({0, {S[0], S[0]}}); rep(i, 0, j) for (; j = (j + 1) % n) { res = max(res, {{S[i] - S[j]}.dist2(), {S[i], S[j]}}); if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0) break; } return res.second; }</pre>	

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.
Time: $\mathcal{O}(\log N)$

"Point.h", "sideOf.h", "OnSegment.h"	71446b, 14 lines
<pre>typedef Point<ll> P; bool inHull(const vector<P>& l, P p, bool strict = true) { int a = 1, b = sz(l) - 1, r = !strict; if (sz(l) < 3) return r && onSegment(l[0], l.back(), p); if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b); if (sideOf(l[0], l[a], p) >= r sideOf(l[0], l[b], p) <= -r) return false; while (abs(a - b) > 1) { int c = (a + b) / 2; (sideOf(l[0], l[c], p) > 0 ? b : a) = c; } return sgn(l[a].cross(l[b], p)) < r; }</pre>	

LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: $\bullet(-1, -1)$ if no collision, $\bullet(i, -1)$ if touching the corner i , $\bullet(i, i)$ if along side $(i, i + 1)$, $\bullet(i, j)$ if crossing sides $(i, i + 1)$ and $(j, j + 1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i, i + 1)$. The points are returned in the same order as the line hits the polygon. extrVertex returns the point of a hull with the max projection onto a line.
Time: $\mathcal{O}(\log n)$

"Point.h"	7cf45b, 39 lines
<pre>#define cmp(i, j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n])) #define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0 template <class P> int extrVertex(vector<P>& poly, P dir) { int n = sz(poly), lo = 0, hi = n; if (extr(0)) return 0; while (lo + 1 < hi) { int m = (lo + hi) / 2; if (extr(m)) return m; }</pre>	

```
int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
(ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
}
return lo;
}

#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
    int endA = extrVertex(poly, (a - b).perp());
    int endB = extrVertex(poly, (b - a).perp());
    if (cmpL(endA) < 0 || cmpL(endB) > 0)
        return {-1, -1};
    array<int, 2> res;
    rep(i, 0, 2) {
        int lo = endB, hi = endA, n = sz(poly);
        while ((lo + 1) % n != hi) {
            int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
            (cmpL(m) == cmpL(endB) ? lo : hi) = m;
        }
        res[i] = (lo + !cmpL(hi)) % n;
        swap(endA, endB);
    }
    if (res[0] == res[1]) return {res[0], -1};
    if (!cmpL(res[0]) && !cmpL(res[1]))
        switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
            case 0: return {res[0], res[0]};
            case 2: return {res[1], res[1]};
        }
    return res;
}
```

8.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.
Time: $\mathcal{O}(n \log n)$

"Point.h"	ac41a6, 17 lines
<pre>typedef Point<ll> P; pair<P, P> closest(vector<P> v) { assert(sz(v) > 1); set<P> S; sort(all(v), [](P a, P b) { return a.y < b.y; }); pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}}; int j = 0; for (P p : v) { P d{1 + (ll)sqrt(ret.first), 0}; while (v[j].y <= p.y - d.x) S.erase(v[j++]); auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d); for (; lo != hi; ++lo) ret = min(ret, {(*lo - p).dist2(), { *lo, p } }); S.insert(p); } return ret.second; }</pre>	

kdTree.h

Description: KD-tree (2d, can be extended to 3d)

"Point.h"	69ff55, 63 lines
<pre>typedef long long T; typedef Point<T> P; const T INF = numeric_limits<T>::max(); bool on_x(P a, P b) { return a.x < b.x; } bool on_y(P a, P b) { return a.y < b.y; } struct Node { P pt; // if this is a leaf, the single point in it T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; // bounds</pre>	

```
Node *first = 0, *second = 0;

T distance(P p) { // min squared distance to a point
    T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x);
    T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y);
    return (P(x, y) - p).dist2();
}

Node(vector<P>&& vp) : pt(vp[0]) {
    for (P p : vp) {
        x0 = min(x0, p.x); x1 = max(x1, p.x);
        y0 = min(y0, p.y); y1 = max(y1, p.y);
    }
    if (vp.size() > 1) {
        // split on x if width >= height (not ideal...)
        sort(all(vp), x1 - x0 >= y1 - y0 ? on_x : on_y);
        // divide by taking half the array for each child (not
        // best performance with many duplicates in the middle)
        int half = sz(vp)/2;
        first = new Node({vp.begin(), vp.begin() + half});
        second = new Node({vp.begin() + half, vp.end()});
    }
};

struct KDTree {
    Node* root;
    KDTree(const vector<P>& vp) : root(new Node({all(vp)})) {}

    pair<T, P> search(Node *node, P p) {
        if (!node->first) {
            // uncomment if we should not find the point itself:
            // if (p == node->pt) return {INF, P()};
            return make_pair((p - node->pt).dist2(), node->pt);
        }

        Node *f = node->first, *s = node->second;
        T bfirst = f->distance(p), bsec = s->distance(p);
        if (bfirst > bsec) swap(bsec, bfirst), swap(f, s);

        // search closest side first, other side if needed
        auto best = search(f, p);
        if (bsec < best.first)
            best = min(best, search(s, p));
        return best;
    }

    // find nearest point to a point, and its squared distance
    // (requires an arbitrary operator< for Point)
    pair<T, P> nearest(P p) {
        return search(root, p);
    }
};
```

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order {t[0][0], t[0][1], t[0][2], t[1][0], ... }, all counter-clockwise.
Time: $\mathcal{O}(n \log n)$

"Point.h"	cefd5, 88 lines
<pre>typedef Point<ll> P; typedef struct Quad* Q; typedef __int128_t lll; // (can be ll if coords are < 2e4) P arb(LLONG_MAX, LLONG_MAX); // not equal to any other point struct Quad { Q rot, o; P p = arb; bool mark;</pre>	

```
P& F() { return r()->p; }
Q& r() { return rot->rot; }
Q prev() { return rot->o->rot; }
Q next() { return r()->prev(); }
} *H;

bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
    ll1 p2 = p.dist2(), A = a.dist2()-p2,
        B = b.dist2()-p2, C = c.dist2()-p2;
    return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B > 0;
}

Q makeEdge(P orig, P dest) {
    Q r = H ? H : new Quad{new Quad{new Quad{new Quad{0}}}};
    H = r->o; r->r()->r() = r;
    rep(i,0,4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r();
    r->p = orig; r->F() = dest;
    return r;
}

void splice(Q a, Q b) {
    swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
}

Q connect(Q a, Q b) {
    Q q = makeEdge(a->F(), b->p);
    splice(q, a->next());
    splice(q->r(), b);
    return q;
}

pair<Q,Q> rec(const vector<P>& s) {
    if (sz(s) <= 3) {
        Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
        if (sz(s) == 2) return { a, a->r() };
        splice(a->r(), b);
        auto side = s[0].cross(s[1], s[2]);
        Q c = side ? connect(b, a) : 0;
        return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
    }

#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
    Q A, B, ra, rb;
    int half = sz(s) / 2;
    tie(ra, A) = rec({all(s) - half});
    tie(B, rb) = rec({sz(s) - half + all(s)});
    while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
        (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
    Q base = connect(B->r(), A);
    if (A->p == ra->p) ra = base->r();
    if (B->p == rb->p) rb = base;

#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
    while (circ(e->dir->F(), H(base), e->F())) { \
        Q t = e->dir; \
        splice(e, e->prev()); \
        splice(e->r(), e->r()->prev()); \
        e->o = H; H = e; e = t; \
    }
    for (;;) {
        DEL(LC, base->r(), o); DEL(RC, base, prev());
        if (!valid(LC) && !valid(RC)) break;
        if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
            base = connect(RC, base->r());
        else
            base = connect(base->r(), LC->r());
    }
    return { ra, rb };
}

vector<P> triangulate(vector<P> pts) {
```

```
sort(all(pts)); assert(unique(all(pts)) == pts.end());
if (sz(pts) < 2) return {};
Q e = rec(pts).first;
vector<Q> q = {e};
int qi = 0;
while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p); \
    q.push_back(c->r()); c = c->next(); } while (c != e); }
ADD; pts.clear();
while (qi < sz(q)) if (!(e = q[qi++])->mark) ADD;
return pts;
}
```

8.5 3D

PolyhedronVolume.h

Description: Magic formula for the volume of a polyhedron. Faces should point outwards.

3058c3, 6 lines

```
template<class V, class L>
double signedPolyVolume(const V& p, const L& trilst) {
    double v = 0;
    for (auto i : trilst) v += p[i.a].cross(p[i.b]).dot(p[i.c]);
    return v / 6;
}
```

Point3D.h

Description: Class to handle points in 3D space. T can be e.g. double or long long.

a94b7c, 31 lines

```
template<class T> struct Point3D {
    typedef Point3D P;
    T x, y, z;
    explicit Point3D(T x=0, T y=0, T z=0) : x(x), y(y), z(z) {}
    bool operator<(P p) const {
        return tie(x, y, z) < tie(p.x, p.y, p.z); }
    bool operator==(P p) const {
        return tie(x, y, z) == tie(p.x, p.y, p.z); }
    P operator+(P p) const { return P(x+p.x, y+p.y, z+p.z); }
    P operator-(P p) const { return P(x-p.x, y-p.y, z-p.z); }
    P operator*(T d) const { return P(x*d, y*d, z*d); }
    P operator/(T d) const { return P(x/d, y/d, z/d); }
    T dot(P p) const { return x*p.x + y*p.y + z*p.z; }
    P cross(P p) const {
        return P(y*p.z - z*p.y, z*p.x - x*p.z, x*p.y - y*p.x);
    }
    T dist2() const { return x*x + y*y + z*z; }
    double dist() const { return sqrt((double)dist2()); }
    //Azimuthal angle (longitude) to x-axis in interval [-pi, pi]
    double phi() const { return atan2(y, x); }
    //Zenith angle (latitude) to the z-axis in interval [0, pi]
    double theta() const { return atan2(sqrt(x*x+y*y),z); }
    P unit() const { return *this/(T)dist(); } //makes dist()==1
    //returns unit vector normal to *this and p
    P normal(P p) const { return cross(p).unit(); }
    //returns point rotated 'angle' radians ccw around axis
    P rotate(double angle, P axis) const {
        double s = sin(angle), c = cos(angle); P u = axis.unit();
        return u*dot(u)*(1-c) + (*this)*c - cross(u)*s;
    }
};
```

3dHull.h

Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards.

Time: $\mathcal{O}(n^2)$

5b45fc, 49 lines

```
"Point3D.h"
typedef Point3D<double> P3;
```

```
struct PR {
    void ins(int x) { (a == -1 ? a : b) = x; }
    void rem(int x) { (a == x ? a : b) = -1; }
    int cnt() { return (a != -1) + (b != -1); }
    int a, b;
};

struct F { P3 q; int a, b, c; };

vector<F> hull3d(const vector<P3>& A) {
    assert(sz(A) >= 4);
    vector<vector<PR>> E(sz(A), vector<PR>(sz(A), {-1, -1}));
#define E(x,y) E[f.x][f.y]
    vector<F> FS;
    auto mf = [&](int i, int j, int k, int l) {
        P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
        if (q.dot(A[l]) > q.dot(A[i]))
            q = q * -1;
        F f{q, i, j, k};
        E(a,b).ins(k); E(a,c).ins(j); E(b,c).ins(i);
        FS.push_back(f);
    };
    rep(i,0,4) rep(j,i+1,4) rep(k,j+1,4)
        mf(i, j, k, 6 - i - j - k);

    rep(i,4,sz(A)) {
        rep(j,0,sz(FS)) {
            F f = FS[j];
            if (f.q.dot(A[i]) > f.q.dot(A[f.a])) {
                E(a,b).rem(f.c);
                E(a,c).rem(f.b);
                E(b,c).rem(f.a);
                swap(FS[j--], FS.back());
                FS.pop_back();
            }
        }
        int nw = sz(FS);
        rep(j,0,nw) {
            F f = FS[j];
#define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i, f.c);
            C(a, b, c); C(a, c, b); C(b, c, a);
        }
        for (F& it : FS) if ((A[it.b] - A[it.a]).cross(
            A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c, it.b);
        return FS;
    };
};
```

sphericalDistance.h

Description: Returns the shortest distance on the sphere with radius radius between the points with azimuthal angles (longitude) f1 (ϕ_1) and f2 (ϕ_2) from x axis and zenith angles (latitude) t1 (θ_1) and t2 (θ_2) from z axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. dx*radius is then the difference between the two points in the x direction and d*radius is the total distance between the points.

611f07, 8 lines

```
double sphericalDistance(double f1, double t1,
    double f2, double t2, double radius) {
    double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
    double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
    double dz = cos(t2) - cos(t1);
    double d = sqrt(dx*dx + dy*dy + dz*dz);
    return radius*2*asin(d/2);
}
```

Strings (9)

KMP.h

Description: pi[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -> 0010123). Can be used to find all occurrences of a string.

Time: $\mathcal{O}(n)$

```
vi pi(const string& s) {
    vi p(sz(s));
    rep(i,1,sz(s)) {
        int g = p[i-1];
        while (g && s[i] != s[g]) g = p[g-1];
        p[i] = g + (s[i] == s[g]);
    }
    return p;
}
```

d4375c, 16 lines

```
vi match(const string& s, const string& pat) {
    vi p = pi(pat + '\0' + s), res;
    rep(i,sz(p)-sz(s),sz(p))
        if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
    return res;
}
```

Zfunc.h

Description: z[i] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)

Time: $\mathcal{O}(n)$

```
vi Z(const string& S) {
    vi z(sz(S));
    int l = -1, r = -1;
    rep(i,1,sz(S)) {
        z[i] = i >= r ? 0 : min(r - i, z[i - l]);
        while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
            z[i]++;
        if (i + z[i] > r)
            l = i, r = i + z[i];
    }
    return z;
}
```

ee09e2, 12 lines

Manacher.h

Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down).

Time: $\mathcal{O}(N)$

```
array<vi, 2> manacher(const string& s) {
    int n = sz(s);
    array<vi,2> p = {vi(n+1), vi(n)};
    rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
        int t = r-i+!z;
        if (i<r) p[z][i] = min(t, p[z][l+t]);
        int L = i-p[z][i], R = i+p[z][i]-!z;
        while (L>=1 && R+1<n && s[L-1] == s[R+1])
            p[z][i]++, L--, R++;
        if (R>r) l=L, r=R;
    }
    return p;
}
```

e7ad79, 13 lines

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.

Usage: rotate(v.begin(), v.begin()+minRotation(v), v.end());

Time: $\mathcal{O}(N)$

```
int minRotation(string s) {
```

d07a42, 8 lines

```
int a=0, N=sz(s); s += s;
rep(b,0,N) rep(k,0,N) {
    if (a+k == b || s[a+k] < s[b+k]) {b += max(0, k-1); break;}
    if (s[a+k] > s[b+k]) { a = b; break; }
}
return a;
}
```

SuffixArray.h

Description: Builds suffix array for a string. sa[i] is the starting index of the suffix which is *i*'th in the sorted suffix array. The returned vector is of size $n + 1$, and sa[0] = n. The lcp array contains longest common prefixes for neighbouring strings in the suffix array: lcp[i] = lcp(sa[i], sa[i-1]), lcp[0] = 0. The input string must not contain any nul chars.

Time: $\mathcal{O}(n \log n)$

```
struct SuffixArray {
    vi sa, lcp;
    SuffixArray(string s, int lim=256) { // or vector<int>
        s.push_back(0); int n = sz(s), k = 0, a, b;
        vi x(all(s)), y(n), ws(max(n, lim));
        sa = lcp = y, iota(all(sa), 0);
        for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
            p = j, iota(all(y), n - j);
            rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j;
            fill(all(ws), 0);
            rep(i,0,n) ws[x[i]]++;
            rep(i,1,lim) ws[i] += ws[i - 1];
            for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
            swap(x, y), p = 1, x[sa[0]] = 0;
            rep(i,1,n) a = sa[i - 1], b = sa[i], x[b] =
                (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
        }
        for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
            for (k && k--, j = sa[x[i] - 1];
                s[i + k] == s[j + k]; k++);
    }
};
```

635552, 22 lines

SuffixTree.h

Description: Ukkonen's algorithm for online suffix tree construction. Each node contains indices [l, r] into the string, and a list of child nodes. Suffixes are given by traversals of this tree, joining [l, r] substrings. The root is 0 (has l = -1, r = 0), non-existent children are -1. To get a complete tree, append a dummy symbol – otherwise it may contain an incomplete path (still useful for substring matching, though).

Time: $\mathcal{O}(26N)$

```
struct SuffixTree {
    enum { N = 200010, ALPHA = 26 }; // N ~ 2*maxlen+10
    int toi(char c) { return c - 'a'; }
    string a; // v = cur node, q = cur position
    int t[N][ALPHA], l[N], r[N], p[N], s[N], v=0, q=0, m=2;

    void ukkadd(int i, int c) { suff:
        if (r[v]<=q) {
            if (t[v][c]==-1) { t[v][c]=m; l[m]=i;
                p[m++]=v; v=s[v]; q=r[v]; goto suff; }
            v=t[v][c]; q=l[v];
        }
        if (q==-1 || c==toi(a[q])) q++; else {
            l[m+1]=i; p[m+1]=m; l[m]=l[v]; r[m]=q;
            p[m]=p[v]; t[m][c]=m+1; t[m][toi(a[q])]=v;
            l[v]=q; p[v]=m; t[p[m]][toi(a[l[m]])]=m;
            v=s[p[m]]; q=l[m];
            while (q<r[m]) { v=t[v][toi(a[q])]; q+=r[v]-l[v]; }
            if (q==r[m]) s[m]=v; else s[m]=m+2;
            q=r[v]-(q-r[m]); m+=2; goto suff;
        }
    }
```

aae0b8, 50 lines

```
}

SuffixTree(string a) : a(a) {
    fill(r,r+N,sz(a));
    memset(s, 0, sizeof s);
    memset(t, -1, sizeof t);
    fill(t[1],t[1]+ALPHA,0);
    s[0] = 1; l[0] = l[1] = -1; r[0] = r[1] = p[0] = p[1] = 0;
    rep(i,0,sz(a)) ukkadd(i, toi(a[i]));
}
```

```
// example: find longest common substring (uses ALPHA = 28)
pii best;
int lcs(int node, int i1, int i2, int olen) {
    if (l[node] <= i1 && i1 < r[node]) return 1;
    if (l[node] <= i2 && i2 < r[node]) return 2;
    int mask = 0, len = node ? olen + (r[node] - l[node]) : 0;
    rep(c,0,ALPHA) if (t[node][c] != -1)
        mask |= lcs(t[node][c], i1, i2, len);
    if (mask == 3)
        best = max(best, {len, r[node] - len});
    return mask;
}
static pii LCS(string s, string t) {
    SuffixTree st(s + (char)('z' + 1) + t + (char)('z' + 2));
    st.lcs(0, sz(s), sz(s) + 1 + sz(t), 0);
    return st.best;
}
};
```

Hashing.h

Description: Self-explanatory methods for string hashing.

247f8b, 63 lines

```
const int N = 4e5 + 7;
const int MOD = 1e9+7;
const int PRIME = 331;

ll prime[N];
ll hashes[N];
void init(string s){
    prime[0] = 1;
    for (size_t i = 1; i < N; i++)
        prime[i] = (prime[i-1] * PRIME) % MOD;

    for (size_t i = 0; i < s.size(); i++)
        hashes[i+1] = (hashes[i] * PRIME + s[i] ) % MOD;
}

ll getHash(int l, int r){
    return (hashes[r + 1] - ((hashes[l] * prime[r - l + 1]) %
        MOD) + MOD) % MOD;
}

int compute_hash(string const& s) {
    const int p1 = 31;
    const int m = 1e9 + 9;
    long long hash_valuel = 0;
    long long p_powl = 1;
    for (char c : s) {
        hash_valuel = (hash_valuel + (c - 'A' + 1) * p_powl) %
            m;
        p_powl = (p_powl * p1) % m;
    }
    return hash_valuel;
}
```

```
int combine_hashed_strings(string const& prevStr, int prevHash,
    int newHash){
    return add(prevHash, mul(newHash, power(31, prevStr.size
        ()))));
}

for (char c : s) {
    h1 = (h1 * p1 + (c - 'a' + 1)) % m1;
    h2 = (h2 * p2 + (c - 'a' + 1)) % m2;

    ll ch = (ll(h1) << 32) | h2;

    m[ch]++;
}

struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM = chrono::steady_clock::
            now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
};

gp_hash_table<int, int, custom_hash> mp;
```

AhoCorasick.h
Description: AhoCorasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(−, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries. **Time:** construction takes $\mathcal{O}(26N)$, where N = sum of length of patterns. find(x) is $\mathcal{O}(N)$, where N = length of x. findAll is $\mathcal{O}(NM)$. f35677, 66 lines

```
struct AhoCorasick {
    enum {alpha = 26, first = 'A'}; // change this!
    struct Node {
        // (nmatches is optional)
        int back, next[alpha], start = -1, end = -1, nmatches = 0;
        Node(int v) { memset(next, v, sizeof(next)); }
    };
    vector<Node> N;
    vi backp;
    void insert(string& s, int j) {
        assert(!s.empty());
        int n = 0;
        for (char c : s) {
            int& m = N[n].next[c - first];
            if (m == -1) { n = m = sz(N); N.emplace_back(-1); }
            else n = m;
        }
        if (N[n].end == -1) N[n].start = j;
        backp.push_back(N[n].end);
        N[n].end = j;
        N[n].nmatches++;
    }
    AhoCorasick(vector<string>& pat) : N(1, -1) {
        rep(i, 0, sz(pat)) insert(pat[i], i);
        N[0].back = sz(N);
    }
};
```

```
N.emplace_back(0);

queue<int> q;
for (q.push(0); !q.empty(); q.pop()) {
    int n = q.front(), prev = N[n].back;
    rep(i, 0, alpha) {
        int &ed = N[n].next[i], y = N[prev].next[i];
        if (ed == -1) ed = y;
        else {
            N[ed].back = y;
            (N[ed].end == -1 ? N[ed].end : backp[N[ed].start])
                = N[y].end;
            N[ed].nmatches += N[y].nmatches;
            q.push(ed);
        }
    }
}

vi find(string word) {
    int n = 0;
    vi res; // ll count = 0;
    for (char c : word) {
        n = N[n].next[c - first];
        res.push_back(N[n].end);
        // count += N[n].nmatches;
    }
    return res;
}

vector<vi> findAll(vector<string>& pat, string word) {
    vi r = find(word);
    vector<vi> res(sz(word));
    rep(i, 0, sz(word)) {
        int ind = r[i];
        while (ind != -1) {
            res[i - sz(pat[ind]) + 1].push_back(ind);
            ind = backp[ind];
        }
    }
    return res;
}
};
```

Trie.h
Description: Pointer-based Trie for 26 lowercase letters. Stores insert count (Prefix) and end count (IsEnd) per node. Classical uses (adapt child size for binary tries): - Max/Min XOR pair or query: build binary trie (32 levels), greedily pick opposite bit. - Max XOR with element $\leq m$: offline sort queries and array by m , insert then query. - Count pairs with XOR in $[lo, hi]$: store count metadata, answer $f(hi) - f(lo - 1)$. - Count subarrays with XOR $< K$: use prefix XOR and binary trie. - Dynamic insert/delete: decrement Prefix/IsEnd on deletion path. **Time:** $\mathcal{O}(L)$ per operation where L is the string/integer length. s18cef, 59 lines

```
struct Trie{
    struct Node{
        Node*child[26];
        int IsEnd,Prefix;
        Node(){
            memset(child,0,sizeof child);
            IsEnd=Prefix=0;
        }
    };
    Node*root=new Node();
    void insert(string &s)
    {
        Node*cur=root;
        for(auto it:s)
        {
            int idx=it-'a';
```

```
            if(cur->child[idx]==0)
            {
                cur->child[idx]=new Node();
            }
            cur=cur->child[idx];
            cur->Prefix++;
        }
        cur->IsEnd++;
    }
    bool SearchWord(string &s)
    {
        Node*cur=root;
        for(auto it:s)
        {
            int idx=it-'a';
            if(cur->child[idx]==0)return 0;
            cur=cur->child[idx];
        }
        return cur->IsEnd;
    }
    int CountWord(string &s)
    {
        Node*cur=root;
        for(auto it:s)
        {
            int idx=it-'a';
            if(cur->child[idx]==0)return 0;
            cur=cur->child[idx];
        }
        return cur->IsEnd;
    }
    int CountPrefix(string &s)
    {
        Node*cur=root;
        for(auto it:s)
        {
            int idx=it-'a';
            if(cur->child[idx]==0)return 0;
            cur=cur->child[idx];
        }
        return cur->Prefix;
    }
};
```

Various (10)

10.1 Intervals

IntervalContainer.h
Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive). **Time:** $\mathcal{O}(\log N)$ edce47, 23 lines

```
set<pii>::iterator addInterval(set<pii>& is, int L, int R) {
    if (L == R) return is.end();
    auto it = is.lower_bound({L, R}), before = it;
    while (it != is.end() && it->first <= R) {
        R = max(R, it->second);
        before = it = is.erase(it);
    }
    if (it != is.begin() && (--it)->second >= L) {
        L = min(L, it->first);
        R = max(R, it->second);
        is.erase(it);
    }
    return is.insert(before, {L,R});
}
```

```
void removeInterval(set<pii>& is, int L, int R) {
    if (L == R) return;
    auto it = addInterval(is, L, R);
    auto r2 = it->second;
    if (it->first == L) is.erase(it);
    else (int&)it->second = L;
    if (R != r2) is.emplace(R, r2);
}
```

IntervalCover.h

Description: Compute indices of smallest set of intervals covering another interval. Intervals should be [inclusive, exclusive). To support [inclusive, inclusive], change (A) to add || R.empty(). Returns empty set on failure (or if G is empty).
Time: $\mathcal{O}(N \log N)$

```
template<class T>
vi cover(pair<T, T> G, vector<pair<T, T>> I) {
    vi S(sz(I)), R;
    iota(all(S), 0);
    sort(all(S), [&](int a, int b) { return I[a] < I[b]; });
    T cur = G.first;
    int at = 0;
    while (cur < G.second) { // (A)
        pair<T, int> mx = make_pair(cur, -1);
        while (at < sz(I) && I[S[at]].first <= cur) {
            mx = max(mx, make_pair(I[S[at]].second, S[at]));
            at++;
        }
        if (mx.second == -1) return {};
        cur = mx.first;
        R.push_back(mx.second);
    }
    return R;
}
```

ConstantIntervals.h

Description: Split a monotone function on [from, to) into a minimal set of half-open intervals on which it has the same value. Runs a callback g for each such interval.
Usage: constantIntervals(0, sz(v), [&](int x){return v[x];}, [&](int lo, int hi, T val){...});
Time: $\mathcal{O}(k \log \frac{n}{k})$

```
template<class F, class G, class T>
void rec(int from, int to, F& f, G& g, int& i, T& p, T q) {
    if (p == q) return;
    if (from == to) {
        g(i, to, p);
        i = to; p = q;
    } else {
        int mid = (from + to) >> 1;
        rec(from, mid, f, g, i, p, f(mid));
        rec(mid+1, to, f, g, i, p, q);
    }
}

template<class F, class G>
void constantIntervals(int from, int to, F f, G g) {
    if (to <= from) return;
    int i = from; auto p = f(i), q = f(to-1);
    rec(from, to-1, f, g, i, p, q);
    g(i, to, q);
}
```

10.2 Misc. algorithms

TernarySearch.h

Description: Find the smallest i in $[a, b]$ that maximizes $f(i)$, assuming that $f(a) < \dots < f(i) \geq \dots \geq f(b)$. To reverse which of the sides allows non-strict inequalities, change the $<$ marked with (A) to $<=$, and reverse the loop at (B). To minimize f , change it to $>$, also at (B).
Usage: int ind = ternSearch(0,n-1,[&](int i){return a[i];});
Time: $\mathcal{O}(\log(b-a))$

```
template<class F>
int ternSearch(int a, int b, F f) {
    assert(a <= b);
    while (b - a >= 5) {
        int mid = (a + b) / 2;
        if (f(mid) < f(mid+1)) a = mid; // (A)
        else b = mid+1;
    }
    rep(i,a+1,b+1) if (f(a) < f(i)) a = i; // (B)
    return a;
}
```

LIS.h

Description: Compute indices for the longest increasing subsequence.
Time: $\mathcal{O}(N \log N)$

```
template<class I> vi lis(const vector<I>& S) {
    if (S.empty()) return {};
    vi prev(sz(S));
    typedef pair<I, int> p;
    vector<p> res;
    rep(i,0,sz(S)) {
        // change 0 -> i for longest non-decreasing subsequence
        auto it = lower_bound(all(res), p{S[i], 0});
        if (it == res.end()) res.emplace_back(), it = res.end()-1;
        *it = {S[i], i};
        prev[i] = it == res.begin() ? 0 : (it-1)->second;
    }
    int L = sz(res), cur = res.back().second;
    vi ans(L);
    while (L--) ans[L] = cur, cur = prev[cur];
    return ans;
}
```

FastKnapsack.h

Description: Given N non-negative integer weights w and a non-negative target t , computes the maximum $S \leq t$ such that S is the sum of some subset of the weights.
Time: $\mathcal{O}(N \max(w_i))$

```
int knapsack(vi w, int t) {
    int a = 0, b = 0, x;
    while (b < sz(w) && a + w[b] <= t) a += w[b++];
    if (b == sz(w)) return a;
    int m = *max_element(all(w));
    vi u, v(2*m, -1);
    v[a+m-t] = b;
    rep(i,b,sz(w)) {
        u = v;
        rep(x,0,m) v[x+w[i]] = max(v[x+w[i]], u[x]);
        for (x = 2*m; --x > m;) rep(j, max(0,u[x]), v[x])
            v[x-w[j]] = max(v[x-w[j]], j);
    }
    for (a = t; v[a+m-t] < 0; a--);
    return a;
}
```

10.3 Dynamic programming

KnuthDP.h

Description: When doing DP on intervals: $a[i][j] = \min_{i < k < j} (a[i][k] + a[k][j]) + f(i, j)$, where the (minimal) optimal k increases with both i and j , one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j-1]$ and $p[i+1][j]$. This is known as Knuth DP. Sufficient criteria for this are if $f(b, c) \leq f(a, d)$ and $f(a, c) + f(b, d) \leq f(a, d) + f(b, c)$ for all $a \leq b \leq c \leq d$. Consider also: LineContainer (ch. Data structures), monotone queues, ternary search.
Time: $\mathcal{O}(N^2)$

DivideAndConquerDP.h

Description: Given $a[i] = \min_{lo(i) \leq k < hi(i)} (f(i, k))$ where the (minimal) optimal k increases with i , computes $a[i]$ for $i = L..R-1$.
Time: $\mathcal{O}((N + (hi-lo)) \log N)$

```
struct DP { // Modify at will:
    int lo(int ind) { return 0; }
    int hi(int ind) { return ind; }
    ll f(int ind, int k) { return dp[ind][k]; }
    void store(int ind, int k, ll v) { res[ind] = pii(k, v); }

    void rec(int L, int R, int LO, int HI) {
        if (L >= R) return;
        int mid = (L + R) >> 1;
        pair<ll, int> best(LLONG_MAX, LO);
        rep(k, max(LO, lo(mid)), min(HI, hi(mid)))
            best = min(best, make_pair(f(mid, k), k));
        store(mid, best.second, best.first);
        rec(L, mid, LO, best.second+1);
        rec(mid+1, R, best.second, HI);
    }
    void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX); }
};
```

10.4 Debugging tricks

- signal(SIGSEGV, [](int) { _Exit(0); }); converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). _GLIBCXX_DEBUG failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).
- feenableexcept(29); kills the program on NaNs (1), 0-divs (4), infinities (8) and denormals (16).

10.5 Optimization tricks

__builtin_ia32_ldmxcsr(40896); disables denormals (which make floats 20x slower near their minimum value).

10.5.1 Bit hacks

- $x \& -x$ is the least bit in x .
- for (int x = m; x;) { --x &= m; ... } loops over all subset masks of m (except m itself).
- $c = x \& -x$, $r = x + c$; $((r^x) >> 2) / c$ | r is the next number after x with the same number of bits set.
- rep(b,0,K) rep(i,0,(1 << K)) if (i & 1 << b) D[i] += D[i^(1 << b)]; computes all sums of subsets.

10.5.2 Pragmas

- **#pragma** GCC optimize ("Ofast") will make GCC auto-vectorize loops and optimizes floating points better.
- **#pragma** GCC target ("avx2") can double performance of vectorized code, but causes crashes on old machines.
- **#pragma** GCC optimize ("trapv") kills the program on integer overflows (but is really slow).

FastMod.h

Description: Compute $a\%b$ about 5 times faster than usual, where b is constant but not known at compile time. Returns a value congruent to a (mod b) in the range $[0, 2b)$.

```
751a02, 8 lines
typedef unsigned long long ull;
struct FastMod {
    ull b, m;
    FastMod(ull b) : b(b), m(-1ULL / b) {}
    ull reduce(ull a) { // a % b + (0 or b)
        return a - (ull)((__uint128_t(m) * a) >> 64) * b;
    }
};
```

FastInput.h

Description: Read an integer from stdin. Usage requires your program to pipe in input from file.
Usage: ./a.out < input.txt
Time: About 5x as fast as cin/scanf.

```
7b3c70, 17 lines
inline char gc() { // like getchar()
    static char buf[1 << 16];
    static size_t bc, be;
    if (bc >= be) {
        buf[0] = 0, bc = 0;
        be = fread(buf, 1, sizeof(buf), stdin);
    }
    return buf[bc++]; // returns 0 on EOF
}

int readInt() {
    int a, c;
    while ((a = gc()) < 40);
    if (a == '-') return -readInt();
    while ((c = gc()) >= 48) a = a * 10 + c - 48;
    return a - 48;
}
```

BumpAllocator.h

Description: When you need to dynamically allocate many objects and don't care about freeing them. "new X" otherwise has an overhead of something like 0.05us + 16 bytes per allocation.

```
745db2, 8 lines
// Either globally or in a single class:
static char buf[450 << 20];
void* operator new(size_t s) {
    static size_t i = sizeof buf;
    assert(s < i);
    return (void*)&buf[i -= s];
}
void operator delete(void*) {}
```

SmallPtr.h

Description: A 32-bit pointer that points into BumpAllocator memory.

```
2dd6c9, 10 lines
template<class T> struct ptr {
    unsigned ind;
```

```
ptr(T* p = 0) : ind(p ? unsigned((char*)p - buf) : 0) {
    assert(ind < sizeof buf);
}
T& operator*() const { return *(T*)(buf + ind); }
T* operator->() const { return &*this; }
T& operator[](int a) const { return (&*this)[a]; }
explicit operator bool() const { return ind; }
};
```

BumpAllocatorSTL.h

Description: BumpAllocator for STL containers.
Usage: vector<vector<int, small<int>>> ed(N);

```
bb6d4, 14 lines
char buf[450 << 20] alignas(16);
size_t buf_ind = sizeof buf;

template<class T> struct small {
    typedef T value_type;
    small() {}
    template<class U> small(const U&) {}
    T* allocate(size_t n) {
        buf_ind -= n * sizeof(T);
        buf_ind &= 0 - alignof(T);
        return (T*)(buf + buf_ind);
    }
    void deallocate(T*, size_t) {}
};
```

SIMD.h

Description: Cheat sheet of SSE/AVX intrinsics, for doing arithmetic on several numbers at once. Can provide a constant factor improvement of about 4, orthogonal to loop unrolling. Operations follow the pattern `__mm(256)?name.(si(128|256)|epi(8|16|32|64)|pd|ps)".` Not all are described here; grep for `__mm_` in `/usr/lib/gcc/*/4.9/include/` for more. If AVX is unsupported, try 128-bit operations, "emmintrin.h" and `#define __SSE__` and `__MMX__` before including it. For aligned memory use `__mm_malloc(size, 32)` or `int buf[N] alignas(32)`, but prefer `loadu/storeu`.

```
551b82, 43 lines
#pragma GCC target ("avx2") // or sse4.1
#include "immintrin.h"

typedef __m256i mi;
#define L(x) __mm256_loadu_si256((mi*)&(x))

// High-level/specific methods:
// load(u)?_si256, store(u)?_si256, setzero_si256, __mm_malloc
// blendv_(epi8|ps|pd)(z?y:x), movemask_epi8 (hibits of bytes)
// i32gather_epi32(addr, x, 4): map addr[] over 32-b parts of x
// sad_epu8: sum of absolute differences of u8, outputs 4xi64
// maddubs_epi16: dot product of unsigned i7's, outputs 16xi15
// madd_epi16: dot product of signed i16's, outputs 8xi32
// extractf128_si256(, i) (256->128), cvtsi128_si32 (128->lo32)
// permute2f128_si256(x,x,1) swaps 128-bit lanes
// shuffle_epi32(x, 3*64+2*16+1*4+0) == x for each lane
// shuffle_epi8(x, y) takes a vector instead of an imm

// Methods that work with most data types (append e.g. _epi32):
// set1, blend (i8?x:y), add, adds (sat.), mullo, sub, and/or,
// andnot, abs, min, max, sign(1,x), cmp(gt|eq), unpack(lo|hi)
```

```
int sumi32(mi m) { union {int v[8]; mi m;} u; u.m = m;
    int ret = 0; rep(i,0,8) ret += u.v[i]; return ret; }
mi zero() { return __mm256_setzero_si256(); }
mi one() { return __mm256_set1_epi32(-1); }
bool all_zero(mi m) { return __mm256_testz_si256(m, m); }
bool all_one(mi m) { return __mm256_testc_si256(m, one()); }

ll example_filteredDotProduct(int n, short* a, short* b) {
```

```
int i = 0; ll r = 0;
mi zero = __mm256_setzero_si256(), acc = zero;
while (i + 16 <= n) {
    mi va = L(a[i]), vb = L(b[i]); i += 16;
    va = __mm256_and_si256(__mm256_cmpgt_epil6(vb, va), va);
    mi vp = __mm256_madd_epil6(va, vb);
    acc = __mm256_add_epi64(__mm256_unpacklo_epi32(vp, zero),
        __mm256_add_epi64(acc, __mm256_unpackhi_epi32(vp, zero)));
}
union {ll v[4]; mi m;} u; u.m = acc; rep(i,0,4) r += u.v[i];
for (;i<n;++i) if (a[i] < b[i]) r += a[i]*b[i]; // <- equiv
return r;
}
```


Techniques (A)

techniques.txt	159 lines
Recursion	
Divide and conquer	
Finding interesting points in N log N	
Algorithm analysis	
Master theorem	
Amortized time complexity	
Greedy algorithm	
Scheduling	
Max contiguous subvector sum	
Invariants	
Huffman encoding	
Graph theory	
Dynamic graphs (extra book-keeping)	
Breadth first search	
Depth first search	
* Normal trees / DFS trees	
Dijkstra's algorithm	
MST: Prim's algorithm	
Bellman-Ford	
Konig's theorem and vertex cover	
Min-cost max flow	
Lovasz toggle	
Matrix tree theorem	
Maximal matching, general graphs	
Hopcroft-Karp	
Hall's marriage theorem	
Graphical sequences	
Floyd-Warshall	
Euler cycles	
Flow networks	
* Augmenting paths	
* Edmonds-Karp	
Bipartite matching	
Min. path cover	
Topological sorting	
Strongly connected components	
2-SAT	
Cut vertices, cut-edges and biconnected components	
Edge coloring	
* Trees	
Vertex coloring	
* Bipartite graphs (=> trees)	
* 3^n (special case of set cover)	
Diameter and centroid	
K'th shortest path	
Shortest cycle	
Dynamic programming	
Knapsack	
Coin change	
Longest common subsequence	
Longest increasing subsequence	
Number of paths in a dag	
Shortest path in a dag	
Dynprog over intervals	
Dynprog over subsets	
Dynprog over probabilities	
Dynprog over trees	
3^n set cover	
Divide and conquer	
Knuth optimization	
Convex hull optimizations	
RMQ (sparse table a.k.a 2^k-jumps)	
Bitonic cycle	
Log partitioning (loop over most restricted)	
Combinatorics	

Computation of binomial coefficients	
Pigeon-hole principle	
Inclusion/exclusion	
Catalan number	
Pick's theorem	
Number theory	
Integer parts	
Divisibility	
Euclidean algorithm	
Modular arithmetic	
* Modular multiplication	
* Modular inverses	
* Modular exponentiation by squaring	
Chinese remainder theorem	
Fermat's little theorem	
Euler's theorem	
Phi function	
Frobenius number	
Quadratic reciprocity	
Pollard-Rho	
Miller-Rabin	
Hensel lifting	
Vieta root jumping	
Game theory	
Combinatorial games	
Game trees	
Mini-max	
Nim	
Games on graphs	
Games on graphs with loops	
Grundy numbers	
Bipartite games without repetition	
General games without repetition	
Alpha-beta pruning	
Probability theory	
Optimization	
Binary search	
Ternary search	
Unimodality and convex functions	
Binary search on derivative	
Numerical methods	
Numeric integration	
Newton's method	
Root-finding with binary/ternary search	
Golden section search	
Matrices	
Gaussian elimination	
Exponentiation by squaring	
Sorting	
Radix sort	
Geometry	
Coordinates and vectors	
* Cross product	
* Scalar product	
Convex hull	
Polygon cut	
Closest pair	
Coordinate-compression	
Quadtrees	
KD-trees	
All segment-segment intersection	
Sweeping	
Discretization (convert to events and sweep)	
Angle sweeping	
Line sweeping	
Discrete second derivatives	
Strings	
Longest common substring	
Palindrome subsequences	

Knuth-Morris-Pratt	
Tries	
Rolling polynomial hashes	
Suffix array	
Suffix tree	
Aho-Corasick	
Manacher's algorithm	
Letter position lists	
Combinatorial search	
Meet in the middle	
Brute-force with pruning	
Best-first (A*)	
Bidirectional search	
Iterative deepening DFS / A*	
Data structures	
LCA (2^k-jumps in trees in general)	
Pull/push-technique on trees	
Heavy-light decomposition	
Centroid decomposition	
Lazy propagation	
Self-balancing trees	
Convex hull trick (wcipeg.com/wiki/Convex_hull_trick)	
Monotone queues / monotone stacks / sliding queues	
Sliding queue using 2 stacks	
Persistent segment tree	